AI 时代的数据中心税

六个业务场景的端到端业务流、税的落点与占比，以及鲲鹏基础加速的发力点

基于 76 篇文献 / 资料笔记（2015–2026）与 INSIGHT_AI_WORKLOAD.md 三轮校正 · 2026-09-20 · 研讨版（前 5 页为执行摘要）

#1 拷贝 **#2 序列化** **#8 tokenize** **#10 调度** **#11 沙箱** **#6 状态驻留**

标注约定：〔已发生〕 / 〔趋势〕；〔实测〕 / 〔推算〕；→ 950 = 鲲鹏 920/950 上现在能做的软件 ·SKU· 生态动作；→ 960 = 2028 鲲鹏 960 及后续的架构动作； [NN] = notes/ 笔记编号

一页结论： AI 化没有消灭数据中心税，而是换品类、加重、并新增五个税种

① 税的形态从 "CPU 周期 " 变成 " 加速器空转 + 存着等 "

fleet 级剖析（[01] 式）看不见它： CPU 欠配时 TTFT 高 1.47–7.11× 但表现为 NPU 空转 [69]； Agent 会话中位 80% 时间 " 活着但空闲 "[57]。这也是找不到 "AI 时代 tax 再刻画 " 论文的原因。

② Agent 是税增长最快的一条线，且工具 / 沙箱执行比模型生成更耗时

真实编码 Agent 请求里工具执行占 59.8%、 LLM 生成 41% [56]； RL rollout 中容器方案的环境准备占 19–29% [76]； 10 个 Agent 应用里 5 个由非 LLM 组件主导时延 [57]。

③ 业界正把 host 税从 " 优化 CPU" 改为 " 绕过 CPU"

NPU 侧采样 [44]、 graph 模式 [60]、 DPU + GPU 持久 kernel [58]。 CPU 在 AI 集群里的价值正向状态层（大 DRAM + UB 池）、沙箱层（隔离 / 密度 / 机密计算）、数据层（KAE + 存储路径）转移—— CloudMatrix 论文 6.1.3 已明说要做 CPU 池化节点 [44]。

老税种换形态

拷贝： KB 级 RPC payload → GB 级 KV cache 搬移
序列化： protobuf → tensor + JSON （MCP/ 工具）
压缩：日志 / 存储 → checkpoint、镜像、 KV （需变换）
网络栈： ms 级请求 / 响应 → 每 token 一条流式小消息
分配： malloc → KV 块表 + 会话状态驻留

新增五个税种

#8 tokenize / detokenize （命中率→ 0.99 时占 TTFT 64%）
#9 采样与约束解码
#10 host 调度 / kernel launch / 同步
#11 沙箱生命周期
#14 O(上下文) 的逐轮重处理

" 税 " 沿用 Kanev 等 [01] 的定义：与业务逻辑无关、跨应用共有、能被 " 不懂模型 " 的硬件单元执行的底层开销。传统 fleet 税 22–27% cycles [01]， RPC 全路径 7.1% [02]。

税 × 业务场景热力图： 14 个税种落在 6 个业务场景的哪里

税种	① 电商交易	② 信息流推荐	③ 智能客服 RAG	④ 编码 Agent	⑤ 训练与后训练平台	⑥ MaaS 推理
#1 拷贝 / 搬移	中 · 实测	**大 · 实测**	中 · 推算	中 · 实测	**大 · 实测**	**大 · 实测**
#2 序列化 / 解析	中 · 实测	中 · 推算	**大 · 实测**	**大 · 实测**	中 · 实测	小 · 推算
#3 压缩	**大 · 实测**	小 · 推算	小 · 推算	小 · 实测	**大 · 实测**	小 · 实测
#4 哈希	小 · 实测	小 · 推算	小 · 推算	小 · 实测	—	小 · 实测
#5 加密	小 · 实测	—	小 · 推算	小 · 推算	**大 · 实测**	小 · 推算
#6 分配 / 状态驻留	**大 · 实测**	**大 · 实测**	中 · 推算	**大 · 实测**	中 · 实测	**大 · 实测**
#7 网络栈 /RPC	**大 · 实测**	中 · 推算	**大 · 实测**	**大 · 实测**	小 · 实测	中 · 推算
#8 tokenize	—	小 · 推算	中 · 推算	**大 · 实测**	中 · 实测	**大 · 实测**
#9 采样 / 约束解码	—	小 · 推算	小 · 推算	小 · 推算	小 · 推算	中 · 实测
#10 调度 /launch	小 · 实测	小 · 推算	小 · 推算	中 · 推算	中 · 实测	**大 · 实测**
#11 沙箱生命周期	—	—	小 · 推算	**大 · 实测**	**大 · 实测**	—
#12 向量距离核	—	**大 · 实测**	**大 · 实测**	中 · 实测	—	—
#13 数据层编解码	—	中 · 推算	—	—	**大 · 实测**	—
#14 O(上下文) 重处理	—	—	小 · 推算	**大 · 实测**	小 · 推算	中 · 实测

等级 = 该税在该场景 " 一次业务请求 / 一个作业 " 里的量级， A （时延 / 进度占比）与 B （CPU 侧资源占比）取较高者：大 ≥ 30%、中 5–30%、小 < 5%、— 不出现；实测 = 有文献数字，推算 = 本文算式或类比。⑤ 含预训练与 RL 后训练。逐格出处见 p32 完整矩阵。

按端到端时延占比排序的痛点榜 Top 8—— 以客户为中心

#	痛点（税）	A： 端到端时延占比（实测为主）	落在哪块硅	谁能吃掉
1	沙箱 / 工具执行（#11）	④ 编码 Agent： 工具执行占请求响应 59.8%、生成 41% [56]； ⑤ RL 后训练：容器环境准备占 rollout 19–29% [76]； 5/10 Agent 应用非 LLM 主导 [57]	CPU 核 + DRAM + 虚拟化	鲲鹏 CPU （隔离 / 密度 / 机密计算）；→960 硬件辅助快照
2	tokenize + O(上下文) 重处理（#8 #14）	⑥ MaaS / ④ Agent： 命中率→ 0.99 时 tokenize 占 TTFT 64% [68]； CPU 欠配 TTFT 1.47–7.11× [69]； 冗余 prefill 3.8× [56]	CPU 单核（BPE 串行）	单核性能 + 核数配比；增量化；前端 CPU 池
3	引擎外编排 RPC / 序列化（#7 #2）	③ 客服 RAG： LLM 调用 30–50% 延迟在引擎外（最坏 >70%） [66]； ④ Agent： runner 残余 18.6% [56]、每会话数十 MB 序列化 [57]； ② 推荐：级联多阶段传递（OneRec 单阶段后 OPEX 10.6% [77]）	CPU 核	REPORT 第 5 章方案； bRPC over UB； 结构化数据变换引擎
4	向量检索 / embedding 表（#12 #1）	③ 客服 RAG： 检索占端到端 <1%–86% [52]、非 LLM 模块 >50% [73]； ② 推荐： ANN 召回 + DLRM embedding 表 gather 内存主导 [72]	内存带宽 + 容量（18 GB/s/ 核、80% 带宽）	SKU 内存带宽 / 通道； UB 内存池作第二层内存 [65]
5	训练数据层与 checkpoint （#13 #3 #1）	⑤ 训练平台： checkpoint 200 s/ 次 [63]、同步拖慢 43% [62]； 评估作业 29.5% 在加载 + tokenize [62]； 数据层 DSI 功耗可超训练 [53]	host CPU + 内存带宽（加载占 40% CPU + 55% 带宽 [53]）	KAE sec/zip in-line； DSA 式搬移；大 DRAM
6	host 调度 / launch （#10）	⑥ MaaS： vLLM V0 每步 CPU ~50% （A100/8B） [49]； 昇腾 graph dispatch 0.6–0.8 ms [44]； 共置干扰 TTFT 139× [58]	CPU 单核 + TLB/LLC + CPU–NPU 链路	微架构（TLB/LLC/IPC）； UB 提交队列； host 与沙箱分池
7	KV 搬移与状态层（#1 #6）	⑥ MaaS： 逐层流水后 KV 传输 <7%、第二 token +16.5% （串行 +64%） [46]； EMS over UB TTFT −34%/−59% [44]； ④ Agent： TTL 驱逐占 31.5% 费用 [57]	内存 + 互连	UB 内存语义（已在生产 [44][60]）；大 DRAM； zswap 式压缩
8	传统 datacenter tax 延续	① 电商交易： RPC 延迟税均值 2%、尾部 96% [02]； sidecar P99 +180% [18]； cycles 22–27% [01]——③④⑥ 的编排层同样承担	CPU 核	REPORT 第 5–6 章（KAE 压缩 / 序列化、 DPA 作 DSA）

排序依据：该税在所属业务场景一次业务请求里的时延占比（取实测的高位值）；同一税种在不同场景占比不同，取影响客户最大的场景（①–⑥ 见 p8）。 B 口径（CPU 周期 / 内存 GB / 互连字节）的排序见 p33。

鲲鹏基础加速的发力点：软件 S1–S11 （→ 950 ）与硬件 H1–H12 （→ 960 / SKU ）

软件发力点（→ 950 ，现在就能做）

S1 前端友好的代码布局 · #7 #2 #3 · ①③④⑥
PGO/BOLT、text 大页、热路径函数重排，针对 RPC 库 / 序列化 / 内核栈 / 网关这些 " 税代码 "

S2 零拷贝与二进制序列化 · #2 #1 #14 · ①②③④
zBuffer / FlatBuffers / Arrow 直接成为 bRPC-UB 的 SGE； embedding 用 fp16 二进制而非 FP32 JSON； messages 增量序列化

S3 用户态 / 内核旁路网络 + UB 消息语义 · #7 · ①③④⑥
bRPC over UB 的小消息 slab 修复（U1）、RM 默认化（U2）、UM 自举建连（U8）、io_uring、busy-poll、Homa 式接收端驱动；流式 token 批量帧 + kTLS （U10）

S4 分配器与大页 · #6 #10 · ①④⑥
mimalloc / TCMalloc 调优（Arm 上 vLLM 2.3×）、THP + text 大页、KV 块表；缓解 Python 栈碎片化地址空间的页表遍历

S5 增量 tokenize 与前端池 · #8 #14 · ⑥④③
TokTier 式增量修复 + tokenized 缓存；每 NPU 前端核数配比；前端 CPU 池 / DPU 承接 tokenize

S6 推理引擎 host 路径 · #10 #9 · ⑥⑤
graph 模式、异步调度、采样 / 元数据上 NPU、绑核与 host– 沙箱分池

S7 状态层软件 · #6 #1 · ⑥④②⑤
实测 Mooncake kunpeng_transport / ubshmem （U9）； EMS 类 KV 池参考架构；空闲会话卸载、TTL 策略、zswap

S8 沙箱软件栈 · #11 · ④⑤
进程级沙箱、快照 + 工作集预取（REAP）、预热池、OverlayFS、virtCCA 机密 VM

S9 KAE 接入 AI 路径 · #3 #5 #13 · ⑤④⑥①
UADK Python/PyTorch 零拷贝绑定； zip 接 checkpoint / 镜像 / zswap； sec in-line 接训练数据层

S10 检索与 embedding 表的 SVE / 带宽调优 · #12 #1 · ②③
faiss / SVS / ScaNN 移植； DLRM gather 与 ANN 的带宽利用率基线

S11 剖析与基准工具链 · 支撑全部
DCPerf 类基线、每 step CPU 时间拆解、共置干扰复现、鲲鹏–昇腾 launch 延迟——决定 H 清单的优先级

硬件发力点（→ 960 进架构 / 定 SKU ）

H1 前端： L1-I / 取指带宽 / BTB / 指令预取 · #7 #2 #3 · ①③④⑥
税代码路径长、二进制数百 MB： 前端受限 15–30%、L1-I MPKI 7–56； 920 L3 取指带宽弱、64 项 BTB

H2 TLB 与页表 · #10 #6 #11 · ⑥④⑤
更大更快的 L2 TLB、页表遍历缓存、大页； Python 栈页表遍历占 LLC stall 85%； 920 L2 TLB 11 周期

H3 单核 IPC 与频率（关键路径核） · #7 #10 #8 · ①⑥④
主频差 9% → 尾延迟差 1.5×； Arm launch 比 x86 慢 17–22%； BPE 串行 2–4 MB/s/ 核

H4 内存系统 · #6 #12 #1 #13 · ②③⑥⑤
通道数 / 带宽（MRDIMM 类）、容量、UB 内存池、内存侧位平面变换 + codec

H5 搬移引擎 · #1 · ①⑤⑥④
DPA （片上搬移加速器）作 DSA、与 UB 内存语义合一；小拷贝也要划算（片外阈值 ≥ 425 B ）

H6 结构化数据变换引擎 · #2 · ①③④②
protobuf + JSON + tensor ，近核 / 片上，异步或链式接口

H7 KAE zip / sec 演进 · #3 #5 #13 · ⑤①④
KB 级小 buffer 固定开销、in-line / DMA 直达不额外过内存、zswap 路径

H8 沙箱硬件辅助 · #11 · ④⑤
stage-2 页表复制、内存清零卸载、快照恢复 DMA、低开销机密计算、SMT 隔离

H9 CPU → NPU 提交路径 · #10 · ⑥⑤
UB 内存语义队列 / doorbell、低延迟 launch （对标 x86 2.3 μs、昇腾 graph dispatch 0.6–0.8 ms ）

H10 SME2 级矩阵扩展 + BF16/INT8 · #12 #9 · ③②⑥
rerank / embedding / draft / 生成式推荐前端上 CPU； 生态（KleidiAI/oneDNN ）已就绪

H11 UB CPU 出口带宽与远端读延迟 · #1 #6 · ⑥②④
CPU x32 lane vs DRAM 带宽哪端先饱和； cache-line 粒度按需取（稀疏注意力 2.1× ）

H12 SKU 配比 · 全部
高密度款（沙箱 / 前端 / 状态层） vs 高单核款（host ）；每 NPU 的 CPU 核数； CPU 池化节点

一句话原则：税落在哪块硅，就在哪块硅上吃掉它——前端 /TLB/ 单核对应老税与 host 税（H1–H3 ），内存系统与 UB 对应状态层与检索（H4 H11 ），搬移 / 变换 / 压缩引擎对应拷贝 / 序列化 / 压缩（H5–H7 ），虚拟化路径对应沙箱（H8 ）。软件先把税测出来、把能软件消掉的消掉，剩下的才是硬件的目标。

UB 专题摘要： RPC 通信时延链上 UB 能吃掉什么—— U1–U10

序列化 / 组包	提交（doorbell）	传输（fabric）	接收通知	核间分发	排队 / 过载	反序列化 / 拷贝	连接与权限
文献量级 zBuffer 25 ns [19]； protobuf 拷贝主体	**文献量级** PCIe doorbell 固定开销；内存互连提交 +14% [12]	**文献量级** eRPC 2.3 µs RTT [31]； Falcon 5.8–8.7 µs [30]； CXL RPC 1.5 vs RDMA 17 µs [29]	**文献量级** 中断唤醒 >50 µs [02]；轮询浪费核	**文献量级** RSS 浪费 30–76% [11]；nanoPU JBSQ [10]	**文献量级** Homa grant [08]；Breakwater 信用 [16]；Aequitas −10% [17]	**文献量级** 网络栈拷贝 >50–60% [33]；DDIO/LLC 占用	**文献量级** RC QP 悬崖 [31][38]； MR 注册走内核 [26]； TLS 建连 1.6 ms [21]
UB 原语 · 状态：有 UMQ 预注册 qbuf + SGE；U6	**UB 原语 · 状态：有** DWQE st64b 直写、 SoC 内无 PCIe； U1	**UB 原语 · 状态：有** RM 无连接 + TPG 多路径 + RTP/CTP； 2.1 µs （超节点）；U2 U3	**UB 原语 · 状态：有 · 待测** JFCE / 消息中断； U5	**UB 原语 · 状态：有 · 待增强** Jetty Group RQ 深度分发； U4	**UB 原语 · 状态：部分** UMQ 立即数授信、 RNR 、SL/VL； U3	**UB 原语 · 状态：有 · 小消息放大** 共享 JFR 、 qbuf 池、 OBMM 共享内存； U6 U7	**UB 原语 · 状态：有 · 默认未开** RM 模式、 MAPT/Token/non-pin； UM 自举； U2 U8

三句话结论

① 传输层已经够用：无连接可靠、松弛排序、多路径、不依赖无损、用户态权限、事件通知、 DWQE—— 对照 Falcon 的功能清单 UB 一项不少，这是相对 RoCE 的实质优势（ REPORT 6.3 ）。② 缺口在 "RPC 感知 " 和 " 默认配置 " ： 按消息的 SL 映射、 CQE 时间戳、可编程 CC 、接收侧资源回传、 Jetty Group 有界队列（ U3 U4 ）； bRPC-UB 默认 RC_TP 、 8 KB 最小块、 TCP 握手建连把 UB 的三个卖点都关掉了（ U1 U2 U8 ）——这些是软件就能改的。③ 最缺的是数据：每核 Mops 、 RTT 、每 Gbps 核数、连接数 vs 延迟、通知→唤醒延迟、远端 Load/Store 延迟， UB 目前公开的只有超节点级 2.1 µs 和 HCOM ≤0.5 µs 软件目标； AI 路径上 EMS/XCCL/Mooncake 后端三条都在生产或主线，但 CPU 侧 URMA 一个延迟数字都没有（ U9 ）。

01

方法：税的定义、两种占比口径、六个业务场景

沿用 Kanev 等（ISCA 2015）的 datacenter tax 定义； A 口径按客户感受的端到端时延， B 口径按鲲鹏卖的 CPU 侧资源

税的定义与判定三问 · 两种占比口径 · 标注体系

什么是 " 税 "

沿用 [01]：与业务逻辑无关、跨应用共有的底层例程。Google fleet 2015： allocation 6–7% 、 compression 5–6% 、 memcpy 4–5% 、 protobuf 3–4% 、 RPC 3% 、 hash 1–2% ，合计 22–27% cycles （内核另占 18–20% ）； RPC 全路径 7.1% [02] 。

AI 税 = 不属于模型前向计算（ NPU 上的 matmul/attention/ 集合通信）、也不属于业务逻辑，而是围绕模型运行、被所有 AI 应用共享的通用底层开销。

判定三问

① 换一个模型、换一个业务，它还在吗?
② 它在框架 / 运行时 / 内核的公共路径上，而不是应用代码里?
③ 一个 " 不懂模型 " 的硬件单元能执行它?
三个都是，就是税。

两种占比口径

A · 端到端时延占比（客户感受）

一次请求 / 一条轨迹的墙钟时间里，税占多少。例： Agent 请求里工具执行 59.8% [56] 、 LLM 调用引擎外 30–50% [66] 、 tokenize 在命中率→ 0.99 时占 TTFT 64% [68] 。用来证明痛点，本 PPT 的排序依据。

B · CPU 侧资源占比（鲲鹏卖的东西）

CPU 周期 / 内存 GB / 内存带宽 / 互连字节里，税占多少。例： [01] 22–27% cycles 、训练节点加载数据占 40% CPU + 55% 内存带宽 [53] 、 2 TB 节点托管 310 个 100K 会话（推算）。用来定发力点。

两者都算；同一税种两口径可能相反（KV 搬移： A 小 B 大）。

标注体系

〔已发生〕	生产系统 / 论文佐证
〔趋势〕	有推理链无硬数据
〔实测〕	文献数字
〔推算〕	本文算式，给出输入
→950	920/950 上现在能做的软件 ·SKU· 生态动作
→960	2028 鲲鹏 960 及后续的架构动作
〔待内部确认〕	鲲鹏 /UB 内部规格，不替代内部数据
[NN]	notes/ 笔记编号（ p41 索引）

六个业务场景：业务步骤、系统形态、业务指标与数据来源

#	业务场景	业务步骤（客户视角）	系统形态	业务指标	数据来源
①	电商交易	浏览 → 加购 → 下单 → 支付 → 履约；每步是一次落到几十到上百个微服务的请求	传统微服务（RPC 76% / MQ 23%）	下单 P99；峰值 QPS；单位请求 CPU 成本	[01][02][03][04][18][21][39]
②	信息流 / 短视频推荐	刷新 → 召回 → 粗排 → 精排 → 重排 → 下发	DLRM 级联（精排 = embedding 表 + MLP，CPU 推理）；AI 化 = 生成式推荐（HSTU / OneRec，transformer）	端到端 100–200 ms；每千次刷新成本；embedding 表容量	[72][77][65]
③	智能客服 / 企业知识问答	提问 → 意图识别 / 改写 → 知识检索 → 生成回答 → 审核 → 转人工 / 工单	RAG + LLM （检索在 CPU，生成走 ⑥）	TTFT 1–2 s；每会话成本；拦截率	[52][66][73][57]
④	代码助手 / 编码 Agent	指令 → 计划 → 读代码 → 改代码 → 跑测试 → 提 PR （循环 8.8 轮）	Agent 在线会话（LLM 走 ⑥ + 工具 + 沙箱）	每任务 4.3 min 、$9.7；工具执行 59.8%；缓存命中 95.7%	[56][57][67][50][51][64][61][74]
⑤	大模型训练与后训练平台	数据准备 → 预训练 → SFT → RL 后训练（rollout + 奖励沙箱）→ 评估 → 发布	训练作业 + RL 循环（host = 数据 + 状态 + 沙箱）	MFU； checkpoint 停顿； rollout 吞吐；每轨迹成本	[53][62][63][71][76]
⑥	模型即服务（MaaS API）	租户请求 → 鉴权 / 限流 / 路由 → 推理 → 流式返回 → 计费；③④的底座	推理服务平台（P/D 分离 + KV 池 + 前端）	每 token 成本； TTFT/TPOT SLO；每 NPU 并发	[44][45][46][47][49][58][59][60][68][69][70][75]

与上一版的差别：推荐单列（DLRM 不是 LLM， "embedding 表查找 " 与 "embedding 模型 " 是两回事）； RL 后训练并入 ⑤（同一平台的后半段）； MaaS 作为独立业务 ⑥，③④ 的 LLM 调用环节写 "→ 走 ⑥ " 。向量数据库并入 ②③。

方法 3/4 · 这一页是 " 场景 → 税表 → 发力点 " 之间的桥；场景页的推导链按这里的机理展开

税的字典 1/2：每个税种为什么发生、卡在哪个硬件资源上、软件和硬件各能做什么

税	出现在	微观机理（为什么发生）	瓶颈资源	软件发力点	硬件发力点	依据
#1 拷贝 / 搬移	①②④⑤⑥	用户态↔内核态 payload 拷贝、序列化本身的拷贝、 NIC DMA 只能到 pinned 缓冲；AI 时代变成 KV/checkpoint 的 GB 级 D2H/H2D 与跨节点搬移、 embedding 表的稀疏 gather。 fleet 里多数拷贝 <512 B，片外卸载只对 ≥ 425 B 划算	核周期 + 内存带宽；小拷贝时是核，大块时是 DMA/ 互连	S2 零拷贝序列化； S3 zerocopy 发送； S7 让 KV 绕过 CPU （ UB/GPUDirect ）	H5 片上搬移引擎（小拷贝也划算）； H11 UB 内存语义； H4 近内存 gather	[01][20][33][44][72]
#2 序列化 / 解析	①②③④⑤	protobuf： 生成代码庞大（每消息 1.3 万行）、字段遍历 + varint + 拷贝， " 拷贝是主体、元数据其次、编码最少 "； JSON： 分支密集的解析、 SIMD 可加速； tensor： 纯拷贝。都是长代码路径 → 前端取指失效、 IPC 低	前端（ i-cache/BTB ） + 拷贝带宽 + 单核	S2 零拷贝 / 二进制格式、增量序列化； S1 代码布局	H6 结构化数据变换引擎（近核、异步 / 链式接口）； H1 前端	[19][25][26][42][57]
#3 压缩	①⑤④	LZ 匹配 = 哈希表随机访问 + 分支密集， IPC 低； RPC 路径是 KB 级同步小 buffer， 95% 字节用弱算法省 CPU； checkpoint/ 镜像 / 日志是大块吞吐口径；浮点张量直接压无效，需位平面变换	核周期；小 buffer 时是调用固定开销；大块时是内存带宽（要 in-line ）	S9 KAE 接入点覆盖 RPC 库 / 文件格式 / checkpoint / zswap	H7 KAE 小 buffer 固定开销压到近核 2× 内、 in-line/DMA 直达； H4 内存侧变换 + codec	[01][02][34][53][70][74]
#4 哈希	①④⑥	CRC/SHA/xxhash 有指令级支持；前缀缓存的块 hash 每 128–512 token 一次，量小	核周期（小）	—	指令级即可，不投引擎	[01][44][45]
#5 加密	①③⑤	TLS record AES-GCM 有 AES 指令，但每 record 固定开销在小 record （流式 token ）时主导；握手是 RSA/ECC ；数据层 TLS 把明文读一遍、密文写一遍——内存带宽放大 3×	小 record ： 核固定开销；大块：内存带宽	S3 kTLS/ 批量帧、会话复用； S9 sec in-line 接数据层	H7 KAE sec in-line 、不额外过内存； NIC/UB 侧 in-line TLS	[02][53][21]
#6 分配 / 状态驻留	①②④⑥⑤	malloc： 指针追逐、 cache/dTLB miss 、锁争用、跨 chiplet 域 2.07×； AI 时代的新形态是 " 状态驻留 "——KV/ 会话 / 沙箱状态占内存但不计算（会话中位 80% 空闲）、 embedding 表 TB 级	分配： cache/TLB 延迟与一致性域；驻留： DRAM 容量、分层与恢复延迟	S4 分配器 / 大页； S7 状态卸载、 TTL 、 zswap	H2 TLB ； H4 内存容量 / 通道 /UB 池； H7 zswap 压缩	[01][35][57][72][74]
#7 内核网络栈 / RPC	①③④⑥	每消息一次 syscall + skb 分配 + 拷贝 + softirq/ 中断 + 上下文切换， 100 Gbps 要 18 核； RPC 库的分发 / 线程 /future ； sidecar 再走一遍；每 token 流式小消息把这项放大；尾延迟来自排队与单线程速度。这些都是长代码路径 → 前端受限	核周期（前端 + 单核）；小消息固定开销	S3 内核旁路 / bRPC over UB （ U1 U2 U6 U8 ） / 接收端驱动； S1 代码布局	H1 前端； H3 单核频率； UB： U1 小消息 /DWQE 、 U2 无连接、 U3 RPC 感知传输、 U4 NI 分发、 U5 通知（ p36–38 ）	[02][32][33][18][06][07]

出现在：① 电商 ② 推荐 ③ 客服 ④ 编码 Agent ⑤ 训练平台 ⑥ MaaS 。 " 瓶颈资源 " 决定发力点落在硬件的哪一块；同一税种小尺寸时是核的事、大尺寸时是 DMA/ 带宽的事。

税的字典 2/2：新增的五个 AI 税种与向量、数据层

税	出现在	微观机理（为什么发生）	瓶颈资源	软件发力点	硬件发力点	依据
#8 tokenize / detokenize	⑥④③⑤②	BPE：正则预分词 + 哈希查找 + 合并优先队列，逐序列串行、分支 / 哈希密集，HF 约 2–4 MB/s/ 核；前端对全文重算（O(上下文)），前缀缓存把 NPU 的活省掉后它成了 TTFT 剩下的最大项	单核 IPC + 前端核数；SIMD 帮助有限	S5 增量 tokenize、tokenized 缓存、前端池 /DPU	H3 单核；H12 每 NPU 前端核数；（GPU/NPU 精确 BPE 是另一条路）	[68][69][58]
#9 采样 / 约束解码	⑥	top-k/top-p 对 15 万词表排序、logit 处理；语法约束的 FSM 掩码；CPU 与 NPU 之间每步同步	核周期 + CPU–NPU 往返	S6 采样 / 元数据搬到 NPU；XGrammar 与推理重叠	H9 提交路径；不投专用引擎	[44][73]
#10 调度 / launch / 同步	⑥⑤④	每个 decode step 回到 host：组 batch、块表、张量准备、Python 元数据、launch 图；页表遍历占 LLC stall 85%（碎片化虚拟地址空间）；共置干扰让 dispatch 间隙 1→40 μs；Arm 核 launch 比 x86 慢 17–22%	TLB/LLC/ 单核 IPC + CPU–NPU 链路延迟；对共置极敏感	S6 graph 模式、异步调度、分池；S4 大页	H2 TLB；H3 IPC；H9 UB 内存语义提交队列	[49][58][59][44]
#11 沙箱生命周期	④⑤	VM 创建 = stage-2 页表建立 + 内存清零 + 设备初始化；快照恢复 = 逐页缺页（执行慢 95%）；gVisor 类 syscall 拦截 2.2×、文件 216×；每任务容器镜像 1.2–2.5 GB；工作集中位 0.8 GB、峰值 28 GB；机密计算 I/O 开销 <29.7%	虚拟化路径（stage-2 TLB、页拷贝 / 清零）、内存容量、syscall 路径	S8 进程级沙箱、快照预取、预热池、OverlayFS	H8 页表复制 / 清零卸载 / 快照 DMA/CCA/SMT 隔离；H2；H12 高密度款	[51][64][61][57][76]
#12 向量距离核	②③④	PQ 码扫描 = SIMD 点积 / 查表，18 GB/s/ 核、80% 内存带宽即上限；图索引 = 每跳一次随机访问；十亿级溢出 DRAM 后 SSD 放大 5.8–7.7×	内存带宽（DRAM 内）与访问粒度（溢出后）	S10 SVE 移植、LVQ/PQ 压缩、索引布局	H4 通道 / 带宽；H11 UB 第二层内存；H10 让 embedding/rerank 上 CPU；不投距离引擎	[52][65][73]
#13 数据层编解码 / 加载	⑤②	解压、解密、过滤、特征变换、JPEG/ 视频解码；训练节点仅加载张量即 40% CPU + 55% 内存带宽（TLS 解密 + Thrift 反序列化 + 网络栈）；预处理功耗可超训练器	内存带宽 + 核；每一步都过一遍内存	S9 KAE in-line 接 DPP/ 加载路径；DPP 与训练节点分离	H7 in-line zip/sec；H5 搬移引擎；H12 CPU 密集节点	[53][62][63]
#14 O(上下文) 逐轮重处理	④⑥③	框架未增量化：每轮把全部 messages 重新 JSON 序列化、网关重新解析、推理前端重新 tokenize、重新分块哈希；每请求 8.8 轮 × ~120K token	单核（与 #2 #8 同）	S5 + S2 增量化（纯软件可消）	无硬件对应；是评估 #2 #8 硬件收益的前提	[56][68][57]

#8 #10 #11 #14 是 "AI 税 " 里最值得单独立项的四个：它们在老的 fleet 剖析里不存在，且各自有明确的硬件资源对应（单核 /TLB/ 虚拟化路径）。

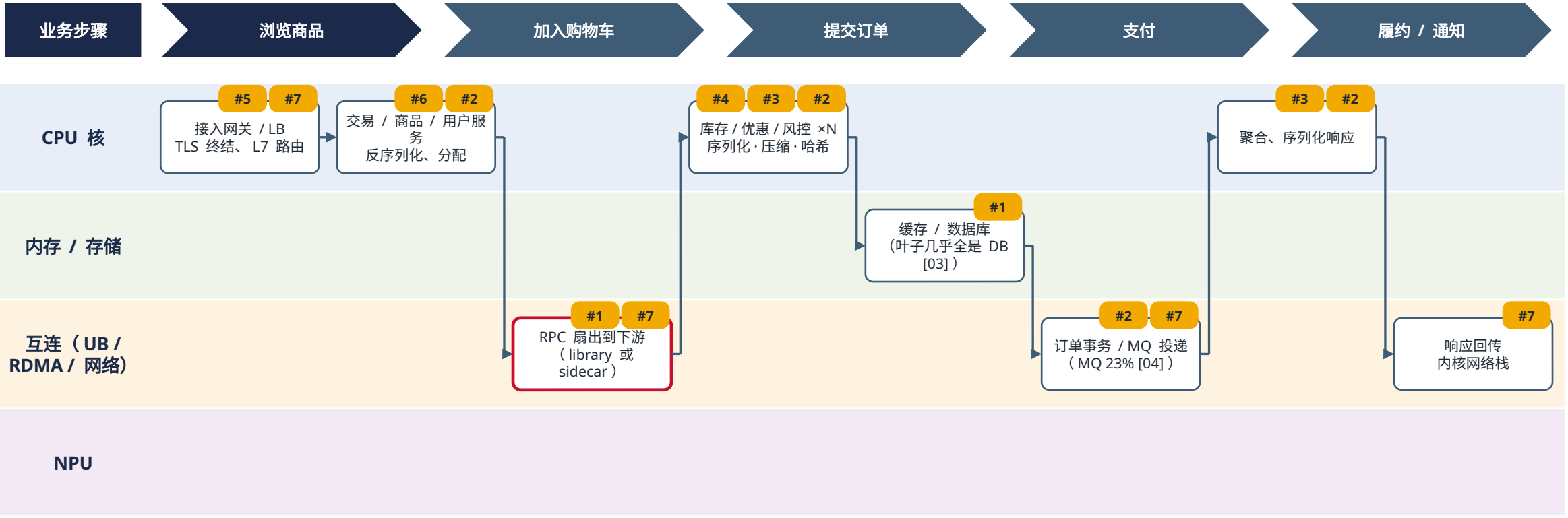
02

六个业务场景：业务流 → 税表 → 推导链

顶部箭头 = 客户看到的业务步骤；泳道 = 系统环节落在哪块硅（CPU 核 / 内存 · 存储 / 互连 / NPU）；琥珀标签 = 税种；红框 = 该场景最重的环节

场景 ① 电商交易：从浏览到履约，每一步都是一次微服务请求

每个业务步骤 = 一次这样的请求（下图为其中一次的系统路径）：中位深 4 、宽 472 的调用树 [03]； RPC 76% / MQ 23% [04]



业务指标

下单 / 支付链路 P99 （客户直接感受）；峰值 QPS （大促）；每请求 CPU 成本 = 请求量 × 每请求 RPC 数 × 每 RPC 成本（REPORT 骨架公式）；生产 RPC 是 ms 级、 KB 级： P90 请求 11.8 KB 、 P99 196 KB [02]

关键刻画

税几乎全落在 CPU 核泳道， NPU 泳道为空——这是 Kanev 六项 + 内核栈的世界。 A 口径 " 平均小、尾部大 "： RPC 延迟税均值 2% 、尾部 96% [02]， sidecar 形态 P99 +180% 、 CPU +273% [18][21]； B 口径稳定： tax 22–27% cycles + 内核 18–20% [01]， RPC 全路径 7.1% [02]。阿里 top 1% 服务占 87% 调用 [39]。

场景 ① 税表： A 端到端时延 / B CPU 侧资源

税	A： 端到端时延占比	B： CPU 侧资源占比	标注
#7 网络栈 / RPC	RPC 延迟税均值 2%，尾部可达 96% [02]； sidecar P99 +180% [18]、 +273% CPU [21]；尾延迟由调用树最慢 1–2 跳决定 [07]	RPC 全路径 7.1% cycles （压缩 3.1 / TCP 1.7 / 序列化 1.2 / 库 1.1 ） [02]；内核 18–20% [01]； 100 Gbps 内核栈需 18 核 [32]	已发生 · 实测
#2 序列化 / 解析	轻逻辑微服务里 RPC 层占 40–90% cycles [13]	protobuf 3–4% [01]；全口径 9.6%、可加速 3.45% [25]；拷贝是主体、元数据其次、编码最少 [19][26]	已发生 · 实测
#3 压缩	（时延占比小，吞吐口径）	5–6% [01]； RPC 路径 3.1% [02]；热点服务 10–50% [34]； 95% 字节用弱算法以省 CPU [01][34]	已发生 · 实测
#1 拷贝 / 搬移	—	memcpy 4–5% [01]；网络栈拷贝 >50–60% [33]；多数拷贝 <512 B [20]	已发生 · 实测
#6 内存分配	—	6–7%，最大单项 [01]； fleet A/B +1.4% 吞吐 [35]	已发生 · 实测
#4 哈希 / #5 加密	—	hash 1–2% [01]；加密 <0.2% （连接复用） [02]	已发生 · 实测
合计	客户感受 = 尾延迟（ CRISP 关键路径 [07] ）	tax 22–27% cycles + 内核 18–20% [01]	—

读法

这条业务是后面五个场景的底座：③④⑥ 的编排层、② 的级联流水线都跑在同样的微服务栈上，这些老税一项都没消失，只是被更大的新税盖住。对电商本身， A 口径的痛点是尾延迟， B 口径是稳定的 1/4 fleet 周期。

场景 ① 推导链：从 " 下单 " 到 " 为什么要加大 L1-I"

业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950 ）	硬件发力点（→ 960 / SKU ）
下单 / 支付 → 网关 + RPC 扇出（每请求 472 次 [03] ）	#7 RPC 全路径 7.1% cycles 、延迟税尾部 96% [02] ； 内核 18–20% [01] ； sidecar P99 +180% [18]	每次 RPC 都走一遍网关 → RPC 库 → 序列化 → 内核栈的长代码路径；二进制数百 MB 、共享库多 → 前端取指失效： Front-end bound 15–30% 、 5–10% cycles 取不到指令、 i-cache 工作集年增 27% [01] ； 生产 L1-I MPKI 7–56 [05] ； tax 代码 IPC 1.0/0.7 vs 核心计算 1.4 [41] 。 920 的 L3 取指带宽 " 接近从 DRAM 取指 " 、 64 项 BTB （ REPORT 6.1 ）正是这条路径的软肋。瓶颈资源：前端（ L1-I 容量、取指带宽、 BTB 、预取）	S1 PGO/BOLT/text 大页把热路径塞进 L1-I ； S3 bRPC over UB / 内核旁路缩短路径本身（ U1 inline+DWQE 、 U2 RM 无连接、 U6 零拷贝接缝）	H1 更大 L1-I 、 L2/L3 取指带宽、 BTB 、指令预取（ DCPerf ： L1-I 减到 1/4 Perf/W −57% 、 cache 微码优化 FB Web +38% [05] ）； U3/U4 UB Controller 做 RPC 感知传输与 NI 分发
下单 → 库存 / 优惠 / 风控 ×N ： 请求 / 响应序列化	#2 protobuf 3–4% [01] ； 轻逻辑服务 RPC 层 40–90% cycles [13]	生成代码庞大（每消息 1.3 万行 [42] ），字段遍历 + varint + 拷贝； " 拷贝是主体、元数据其次、编码最少 " [19][26] 。瓶颈：前端 + 拷贝带宽	S2 零拷贝序列化（ zBuffer 25 ns 、比 Cornflakes 7× [19] ）作 UB 的 SGE	H6 结构化数据变换引擎（近核，异步 / 链式接口， [20][41] 异步比同步好 1.3× ）； H1
各服务 → 压缩响应 / 日志	#3 compression 5–6% [01] ； RPC 路径 3.1% [02] ； 热点服务 10–50% [34]	LZ 匹配 = 哈希表随机访问 + 分支密集， IPC 低； RPC 路径是 KB 级同步小 buffer ， 95% 字节用弱算法省 CPU [01][34] 。瓶颈：核周期 + 调用固定开销（ PCIe 卡慢 5.6× [34] ）	S9 KAE 接入 RPC 库压缩点与文件格式（存储平台压缩占 tax 14–31% [41] ）	H7 KAE 小 buffer 固定开销压到近核 2× 内（ REPORT 6.2 ）
反序列化 / 业务逻辑 → 内存分配	#6 allocation 6–7% ，最大单项 [01] ； TCMalloc 4.3% [35]	指针追逐、 cache/dTLB miss 、锁争用；跨 chiplet 域延迟 2.07× [35] ；线程超订靠自旋 [20] 。瓶颈： cache/TLB 延迟、一致性域	S4 分配器调优 + 大页（ fleet A/B +1.4% 吞吐 [35] ）	H2 TLB ； H4 一致性域 / NUCA （ 950 mesh 待测）
扇出 / 响应 → 内核网络栈拷贝	#1 memcpy 4–5% [01] ； 网络栈拷贝 >50–60% CPU [33] ； 多数拷贝 <512 B [20]	用户↔内核 payload 拷贝、 skb 、 softirq ；小拷贝无法片外卸载（阈值 ≥ 425 B [20] ）。瓶颈：核周期 + 内存带宽	S3 zerocopy 发送 / io_uring / UB 直写（ U6 qbuf 预注册 + non-pin SVA ）	H5 片上搬移引擎（ DPA 作 DSA ，小拷贝也划算）
整条链路 → 尾延迟	主频差 9% → 尾延迟差 1.5× [07] ； 微服务比单体对单线程更敏感 [06]	每跳延迟预算紧，排队 + 单线程速度决定 P99 ；关键路径上只有 1–2 跳真正决定尾部 [07] 。瓶颈：单核频率 /IPC	S11 关键路径识别（ CRISP ），把敏感服务放到快核； U7 同域高频服务对用 OBMM 共享内存 RPC （ 1.5 vs 17 μs ）	H3 关键路径核：高频大核； SMT 对 P99 的影响待测

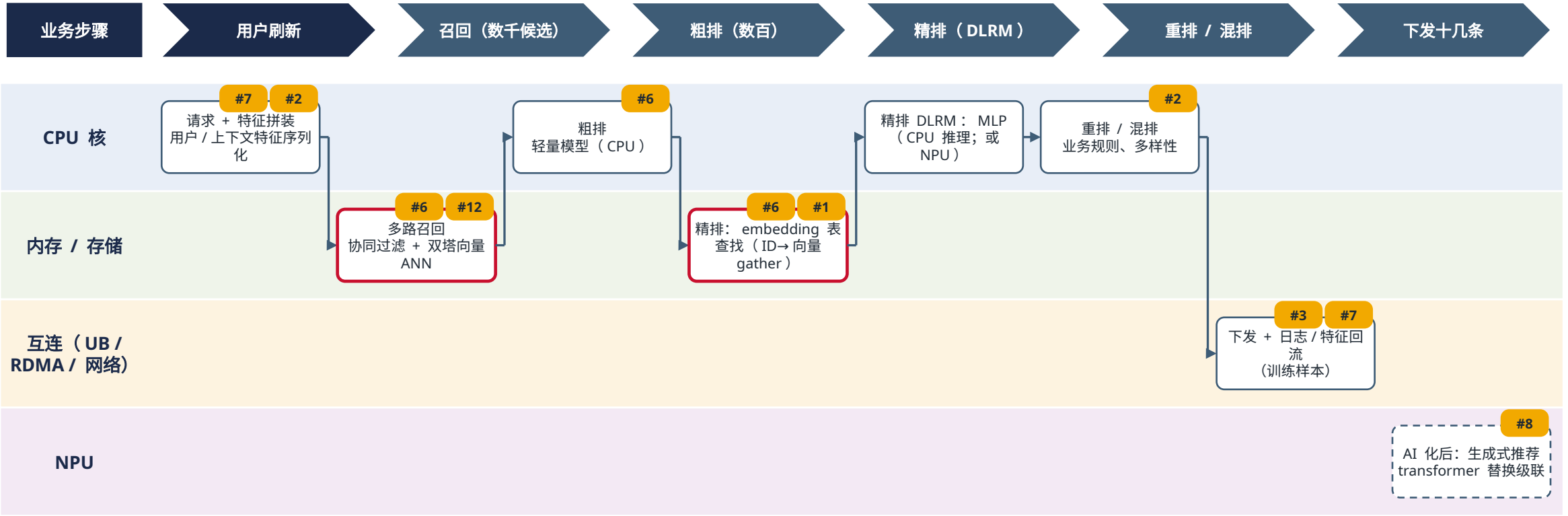
从业务到硅的一条完整推导链

" 下单 " 这个业务动作 = 一次 472 跳的 RPC 树 → 每一跳都要跑网关 /RPC 库 / 序列化 / 内核栈这些与业务无关的公共代码 → 这些代码路径长、二进制大， Google [01] 、 Meta DCPerf [05] 、 Google 大数据平台 [41] 三份独立剖析都指出它们的第一瓶颈是前端取指（ 15–30% 的流水线 slot 在等指令）→ 鲲鹏 920 公开可见的三个微架构弱点（ L3 取指带宽、小 BTB 、慢 L2 TLB ）正好落在这条路径上 → 所以电商这类客户的 fleet 平均最大杠杆是 H1 前端（配合 S1 代码布局），而不是任何加速器；加速器（ H5–H7 ）吃的是拷贝 / 序列化 / 压缩这三项各 3–6% 的具体税。 DCPerf 把这条链闭合： L1-I 只有 1/4 的 ARM SKU Perf/W −57% ；一次 cache 替换微码优化让 FB Web +38% 、 Perf/W +47% ， SPEC 上完全看不到 [05] 。

业务：抖音 / 快手 /Feed 类 · DLRM 级联（不是 LLM ） · AI 化 = 生成式推荐（ HSTU / OneRec ）

场景 ② 信息流 / 短视频推荐：一次刷新的召回 → 排序 → 下发

端到端预算 100–200 ms （业界惯例，推算）； 2019–20 年推荐已占 Facebook AI 推理周期 79% [72]



业务指标

端到端 100–200 ms ；每千次刷新成本； embedding 表容量（ TB 级）与内存带宽；生成式推荐上线后： OneRec 承接 25% QPS 、运营成本降到级联的 10.6% 、 MFU 23.7%/28.8% [77] ； HSTU 1.5T 参数、线上 A/B +12.4% [77]

关键刻画

与 ③ 的 RAG 不同：这里的 "embedding" 是 ID → 向量的表查找（内存 gather ，性能由内存带宽 / 延迟主导， RecNMP 近内存 9.8× [72] ），不是 transformer 编码器。 AI 化改造的真正形态是用 transformer 序列模型替换级联（ HSTU 把排序改成序列转导； OneRec 单阶段合并召回 + 排序），算力主体从 CPU 内存系统迁到 NPU ，但 embedding 表、序列特征拼装 / 序列化、 ANN 召回三项仍留在 CPU/ 内存侧 [77] 。

场景 ② 税表： A 端到端时延 / B CPU 侧资源

税	A： 端到端时延占比	B： CPU 侧资源占比	标注
#1/#6 embedding 表查找（精排）	推荐推理性能由内存受限的稀疏 embedding 操作主导，带宽饱和 [72]；批大小决定延迟 / 吞吐取舍 [72]	推荐占 Facebook AI 推理周期 79%（top 模型 65%）[72]；表 TB 级、不规则 gather；近内存处理内存延迟 9.8×、吞吐 4.2× [72]	已发生 · 实测
#12 ANN 召回	双塔向量召回是带宽型 SIMD 扫描（与 RAG 检索同源 [52]）	PQ 扫描 18 GB/s/ 核、 80% 内存带宽 [52]；溢出 DRAM 后 SSD 放大 5.8–7.7× [65]	已发生 · 实测
#2/#7 级联多阶段传递	每阶段一次特征序列化 + RPC；级联本身就是税： OneRec 单阶段后运营成本 10.6% [77]	特征序列化（推算）；与 ① 的编排层同构	已发生 · 实测（业务级）
#3 日志 / 样本回流	（离线）	日志与训练样本压缩、回流到 ⑤ 的数据层 [53]	已发生 · 实测
生成式推荐（AI 化后）	算力主体迁到 NPU（HSTU 自定义注意力比 FA2 快 5.3–15.2× [77]）；级联的 RPC/ 序列化消失	仍在 CPU/ 内存侧： embedding 表容量、序列特征拼装 /tokenize （#8 新增）、 ANN 召回； host 税形态趋同 ⑥	趋势 · 实测（Meta/ 快手已上线）

读法

A 口径下推荐的税是级联结构本身（多阶段传递）与 embedding 表的内存延迟； B 口径下它是最早、最大的 CPU 侧 AI 负载，对 CPU 的要求是内存容量 + 带宽 + 不规则 gather，与核数关系不大。AI 化之后：级联税被单阶段生成消掉，内存侧的税留下并向 MaaS 的 host 税靠拢。

场景 ② 推导链：从 " 一次刷新 " 到 " 为什么是内存系统而不是核数 "

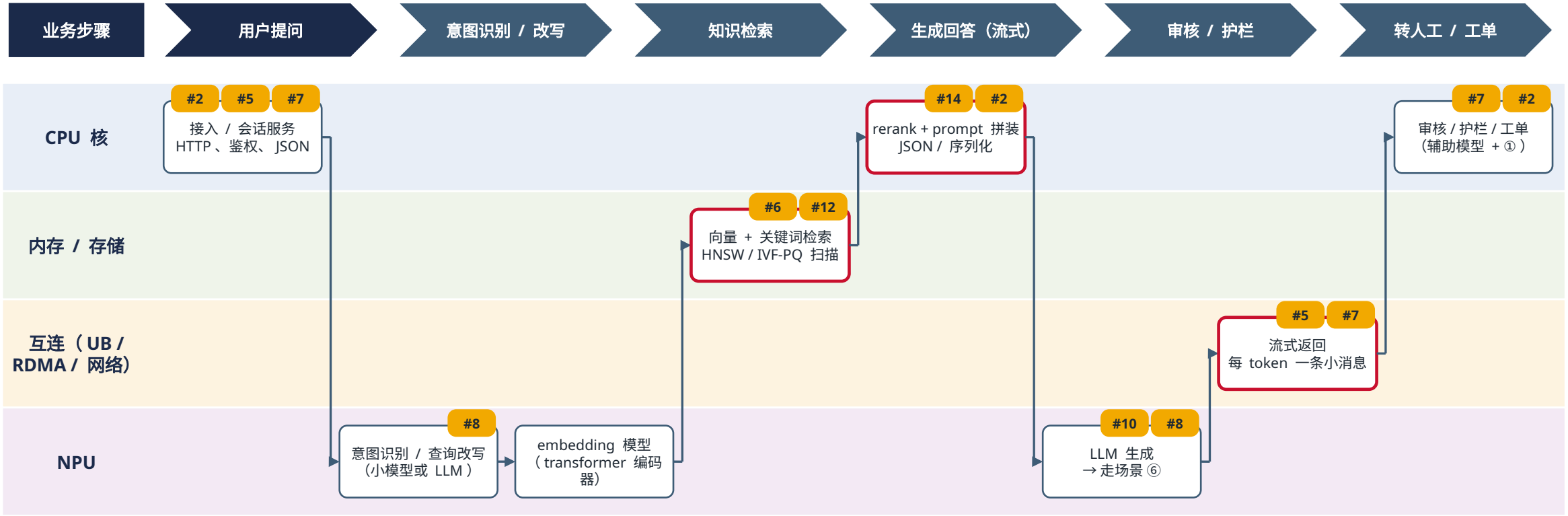
业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950 ）	硬件发力点（→ 960 / SKU ）
精排 → embedding 表查找（稀疏 ID → 向量）	#1/#6 推荐推理由内存受限的稀疏 embedding 操作主导 [72] ；占 AI 推理周期 79% [72]	每个稀疏特征一次随机 gather ，表 TB 级、无局部性，内存带宽饱和后核空转；批大小决定延迟 / 吞吐取舍。瓶颈：内存带宽 + 容量 + 延迟（近内存处理 9.8× 的原因 [72] ）	S10 gather 访存模式优化（批合并、预取）； S7 表溢出时放 UB 内存池	H4 通道数 / 带宽（ MRDIMM 类）、容量； H5 近内存 gather 引擎（ RecNMP 路线）； H11 UB 远端读
召回 → 双塔向量 ANN	#12 带宽型： PQ 扫描 18 GB/s/ 核、 80% 内存带宽 [52]	SIMD 点积 / 查表已达带宽上限；图索引每跳一次随机访问；溢出 DRAM 后 SSD 放大 5.8–7.7× [65] 。瓶颈：带宽 + 访问粒度	S10 SVE 移植、 LVQ/PQ 压缩（ SVS 20.7× 低内存场景 [73] ）	H4 ； H11 UB 作第二层内存（细粒度随机读优于 SSD [65] ）
召回 → 粗排 → 精排 → 重排：级联各阶段的特征传递	#2/#7 级联本身是税： OneRec 单阶段后运营成本降到 10.6% [77]	每阶段一次特征序列化 + RPC ，机理同 ①（前端 + 拷贝 + 内核栈）；候选数从数千到数百到十几，每级都要序列化一遍候选集。瓶颈：前端 + 拷贝	S2 二进制特征格式 / 零拷贝； S1 代码布局；（业务侧：单阶段生成消掉级联）	H6 结构化数据变换引擎； H1 前端
下发 → 日志 / 样本回流	#3 大块离线压缩	日志、曝光 / 点击样本压缩后回流到训练数据层（⑤ 的 #13 ）	S9 KAE zip 接日志 / 样本管道	H7 KAE zip （大块吞吐口径）
AI 化后 → 生成式推荐（ HSTU / OneRec ）	#8 序列 token 化；算力上 NPU （ HSTU 比 FA2 快 5.3–15.2× [77] ）	用户行为序列当 token 序列生成， transformer 在 NPU ； CPU 侧剩下： embedding 表（仍在）、序列特征拼装 + tokenize 、 ANN 召回、前端——与 ⑥ 的 host 税形态趋同。瓶颈：单核 + 前端核数 + 内存	S5 前端池 / 增量 tokenize ； S6 host 路径； S7 表放 UB 池	H10 SME2 让小模型前端上 CPU ； H9 CPU→NPU 提交； H4

从业务到硅的一条完整推导链

" 一次刷新 " 要在 100–200 ms 内完成召回 → 粗排 → 精排 → 重排 → 精排的 DLRM 不是算力问题而是内存问题：每个稀疏特征一次随机 gather 、表 TB 级，带宽一饱和核就空转（ RecNMP 用近内存处理拿到 9.8× 就是证据）→ 所以推荐客户要的鲲鹏是通道数、带宽、容量（ H4 ）和把溢出的表放进 UB 内存池（ H11 ），核数是次要的。 AI 化之后（ HSTU/OneRec 已上线）算力主体迁到 NPU ，级联的 RPC/ 序列化税被单阶段生成消掉（ OPEX 10.6% ），但 embedding 表、序列特征拼装、召回三项仍留在 CPU/ 内存侧——这时推荐变成 ⑥ 的一个租户，它的 CPU 侧要求与 MaaS 前端一样：单核、每 NPU 的前端核数、内存池。

场景 ③ 智能客服 / 企业知识问答：一次提问的检索增强生成

TTFT 目标 1–2 s （推算）；检索占端到端 <1% （长上下文）– 86% （超大规模检索）[52]； LLM 调用 30–50% 延迟在引擎外 [66]



业务指标

TTFT 1–2 s 、每会话成本、拦截率； RAG 问答的非 LLM 模块占端到端 >50% [73]；生产链式应用 30–50% （最坏 >70% ）的延迟在 LLM 引擎之外 [66]；94% 的请求 token 跨用户重复 [66]

关键刻画

这里的 "embedding" 是 transformer 编码器（算力），检索是 CPU 上的带宽型扫描（PQ 18 GB/s/ 核、 80% 内存带宽 [52] ）。payload 从 KB 级结构化数据变成向量（2–16 KB ）、 prompt （16–512 KB ）、 token 流—— LLM 调用的中位 payload 落在传统 RPC 的 P90–P99 区间 [02]；延迟预算从 ~100 ms 变成 1–10 s，传统 µs 级税被稀释，但编排层的 RPC 量与每条 payload 都上升。

场景 ③ 税表： A 端到端时延 / B CPU 侧资源

税	A：端到端时延占比	B： CPU 侧资源占比	标注
#12 向量检索	超大规模检索（640 亿向量、92 TB） 8B 模型短序列 86.3%、长序列 30.9%；长上下文范式 <1% [52]； RAG 问答非 LLM 模块 >50% [73]	ScaNN PQ 扫描 18 GB/s/ 核、内存带宽利用率 ~80%—— 带宽型，SIMD 已近上限 [52]；溢出 DRAM 后 SSD 索引放大 5.8–7.7× [65]	已发生 · 实测
#7/#2 编排 RPC 与序列化	LLM API 调用 30–50% 延迟在引擎外（最坏 >70%），随 prompt 变长而增大 [66]	embedding 以 FP32 JSON 传输时膨胀 2–3×、解析再高一个量级（推算）；一次请求 >20 MB embedding [57]	已发生 · 实测
#7 流式小消息	每 token 一次 JSON+HTTP/2+TLS+syscall；有效载荷占比 2–5% （推算）	10 万并发流 × 50 tok/s = 500 万条 /s，按 [33] 的 2–5 μs/ 条 ≈ 10–25 核（推算）	已发生 · 推算
#2/#14 prompt 拼装	多轮客服会话每轮重新拼装全量上下文（同 ④ 的形态，量级更小）	（推算）	已发生 · 推算
#8/#10/#1 （生成环节）	见场景 ⑥	见场景 ⑥	—

读法

A 口径下最大的两项都不在模型里：检索（配置相关，<1%–86%）和引擎外编排（30–50%）。 B 口径下检索是 " 内存带宽 " 而不是 " 核 " 的事，与 ② 的 ANN 召回同源——对 SKU 的要求是通道数与带宽。压缩 / 加密在这条业务里退居次要（流式 TLS 是每 record 固定开销， KAE 帮不上）。

场景 ③ 推导链：从 " 一次提问 " 到 " 检索要带宽、编排要前端 "

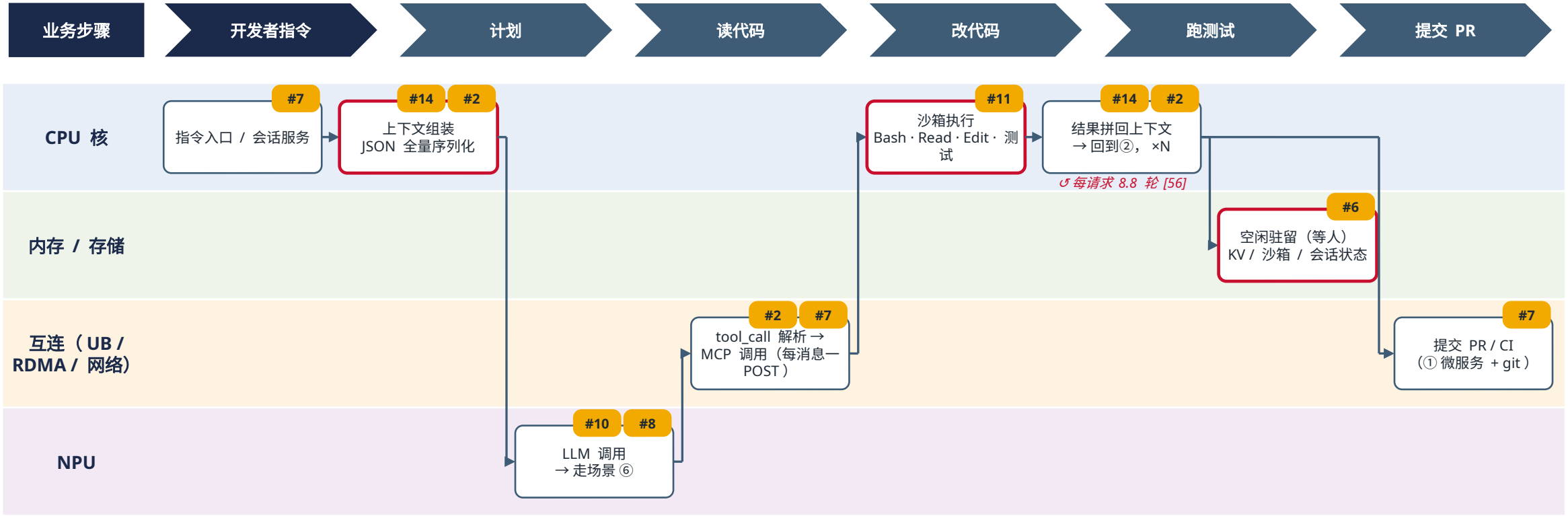
业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950）	硬件发力点（→ 960 / SKU）
知识检索 → 向量 + 关键词检索	#12 检索占端到端 <1%（长上下文）– 86%（超大规模检索）[52]；RAG 问答非 LLM 模块 >50% [73]	PQ 码扫描 = SIMD 点积 / 查表，18 GB/s/ 核、80% 内存带宽即上限 [52]；加速器越快、序列越短、每次检索的查询向量越多，检索占比越高 [52]。瓶颈：内存带宽（DRAM 内）、访问粒度（溢出后）	S10 faiss/SVS/ScaNN 的 SVE 路径与带宽利用率基线	H4 通道 / 带宽；H11 UB 第二层内存 [65]；H10 让 embedding/rerank 模型上 CPU（Neoverse V2 INT8 4.8–6.2× [74]）
编排 → 意图识别 / 检索 / rerank / LLM 的多次调用往返	#7/#2 LLM API 调用 30–50% 延迟在引擎外，最坏 >70%，随 prompt 变长而增大 [66]	客户端–网关–引擎 " 聊天式 " 往返：前一调用输出要绕回客户端再作为下一调用输入；每次往返走一遍网关 /JSON/ 内核栈（机理同 ①，前端 + 拷贝 + 网络栈），且 payload 是 16–512 KB 的 prompt（落在传统 RPC 的 P90–P99 区间 [02]）。瓶颈：前端 + 拷贝带宽 + 内核栈	S3 服务端编排（Parrot 把往返压进引擎内，11.7× [66]）、bRPC over UB（U1 U2）；S2 二进制 /fp16 向量与 prompt 零拷贝（U6）	H1 前端；H6 变换引擎（JSON + tensor）
生成回答 → 流式返回	#7/#5 每 token 一条小消息：JSON+HTTP/2+TLS record+syscall，有效载荷 2–5%（推算）	每 token 一次序列化 + TLS record（小 record 固定开销主导）+ write + 内核栈；10 万流 ≈ 500 万条 /s ≈ 10–25 核（按 [33] 2–5 μs/ 条推算）。瓶颈：内核栈每消息固定开销	S3 批量帧 / kTLS / 用户态栈；U10 超节点内流式 token 走 UMQ 小消息 + 批量帧	H1 前端；NIC/UB 侧 in-line TLS（小 record 时 KAE sec 无效）
多轮对话 → prompt 拼装	#2/#14 每轮全量重编码（同 ④，量级小）	框架未增量化：每轮把全部历史 JSON 序列化、网关解析、前端重 tokenize。瓶颈：单核	S2 + S5 增量化（纯软件可消）	—
审核 / 护栏 / 转人工 → 辅助模型 + 工单微服务	#2/#7 辅助 LLM 调用 + ① 的微服务税	安全护栏、循环检测等辅助调用是 " 控制面税 "（35K 会话额外 2,684 次调用 [57]）；工单 /CRM 是传统微服务，机理同 ①	S1 S3	H1

从业务到硅的一条完整推导链

" 一次提问 " 的时延 = 检索 + 编排往返 + 生成 + 流式 → 检索是 CPU 上的带宽型扫描（PQ 18 GB/s/ 核、80% 带宽），加速器越快它占比越高（<1%→86%），所以客服客户要的第一个东西是内存带宽（H4），第二是让 embedding/rerank 这类小模型留在 CPU 上（H10）→ 编排层的 30–50% 是每次调用绕客户端一圈的网关 /JSON/ 内核栈成本，机理与电商完全相同（前端 + 拷贝），所以 H1/H6 同样适用，而软件上服务端编排（Parrot）能先消掉一半 → 生成环节的 tokenize/ 调度 /KV 税全部计入 ⑥。这条业务里 KAE 现有引擎几乎没有落点：流式 TLS 是小 record，压缩几乎不出现。

场景 ④ 代码助手 / 编码 Agent： 一个开发任务的自治循环

每请求平均 8.8 次 LLM 调用 + 10.8 次工具调用、 4.3 min （中位 38 s， p90 6.4 min ） [56]； 中间步骤由 Agent 自治循环完成



业务指标

每任务 4.3 min 、 $9.70 （中位 $0.61， p99 $178 ） [56]； 工具执行占响应时间 59.8% 、生成 41.0% [56]； 前缀缓存命中 95.7% ，未命中造成 3.8× 冗余 prefill [56]； 成本 59.5% 在前缀读、 11.2% 在输出 [56]

关键刻画

每步中位 119K 前缀 + 875 追加 + 214 输出 token [56]； 前 3 种工具（ Bash/Read/Edit ）占 >80% 调用，即 syscall 与文件 I/O 密集 [56]； 会话中位 80% 时间 " 活着但空闲 "， 5 min TTL 驱逐占 31.5% 费用 [57]； 沙箱工作集中位 0.8 GB 、编译 / 单测峰值 28 GB [57]； 本地 Agent 已用进程级沙箱（ bubblewrap/seccomp/Landlock ），云端仍容器 /microVM [74] 。

场景 ④ 税表： A 端到端时延 / B CPU 侧资源

税	A： 端到端时延占比	B： CPU 侧资源占比	标注
#11 沙箱 / 工具执行	工具执行占请求响应时间 59.8%，生成 41.0% [56]； >1 min 的工具调用只占 4% 却占 85% 工具时间 [56]； 5/10 应用非 LLM 组件主导 [57]	沙箱工作集中位 0.8 GB、峰值 28 GB [57]； gVisor syscall ≥2.2×、 tmpfs open 216× [64]； 快照恢复执行 +95% [64]； virtCCA@ 鲲鹏 920 I/O 开销 <29.7% [61]	已发生 · 实测
#8/#14 tokenize / 重处理	冗余 prefill 3.8×，新鲜追加仅 19% [56]；命中率→ 0.99 时 tokenize 占 TTFT 64% [68]	每步 ~120K token × 8.8 步 ≈ 4.2 MB 文本 / 请求重处理（推算）； TTL 驱逐使 59.4% 会话至少一次重 prefill、占 31.5% 费用 [57]	已发生 · 实测 + 推算
#2/#7 序列化 / 编排 RPC	Codex runner 残余开销 18.6% （均值 1.11 s、 p99 10 s） [56]；引擎外 30–50% [66]	每会话数十 MB 序列化： >20 MB FP32 embedding/ 请求、 >10 MB 文本往返 [57]； MCP = JSON-RPC/ 每消息一 POST/base64 [74]	已发生 · 实测
#6 状态驻留	（不占时延，占资源）	会话中位仅 20% 生命周期在执行 [57]； KV 复用 >80% （聊天 54–62%） [67]；规划期间卸载闲着的沙箱，平均内存 − 4.6× （受控实验） [57]	已发生 · 实测
#1 搬移	（被工具时延盖住）	截图 / embedding / 上下文往返 [57]	已发生 · 实测

读法

A 口径下编码 Agent 的最大税不在模型里：工具 / 沙箱执行（59.8%） > LLM 生成（41%），其次是被人停顿打断的重 prefill 与 tokenize。 B 口径下状态层的形态是 " 存着等 " （80% 空闲）—— 需要便宜、大、几分钟内可恢复的停靠层；序列化量级已到每会话数十 MB。

场景 ④ 推导链：从 " 跑测试 " 到 " 为什么是虚拟化路径而不是加速器 "

业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950 ）	硬件发力点（→ 960 / SKU ）
读代码 / 改代码 / 跑测试 → 沙箱执行（ Bash · Read · Edit · 单测）	#11 工具执行占请求响应 59.8% 、生成 41.0% [56] ； 工作集中位 0.8 GB 、峰值 28 GB [57] ； >1 min 的调用 4% 占 85% 工具时间 [56]	Bash/Read/Edit 占 >80% 工具调用 [56] = 进程创建 + syscall + 文件 I/O ； 隔离层把它们放大：gVisor syscall ≥2.2× 、 tmpfs open 216× 、小文件读 11× [64] ； 快照恢复逐页缺页让执行慢 95% [64] ； 编译 / 单测让内存峰值 28 GB [57] 。瓶颈： syscall/ 虚拟化路径（ stage-2 页表、页拷贝 / 清零）、内存容量、密度	S8 进程级沙箱（本地已如此 [74] ）、快照 + 工作集预取（ 3.7× [64] ）、预热池、OverlayFS 、 virtCCA （ <29.7% [61] ）	H8 stage-2 页表复制、内存清零卸载、快照恢复 DMA 、低开销机密计算、 SMT 隔离； H2 TLB ； H12 高密度款 + 大 DRAM
每轮 → 上下文组装 + LLM 前端 tokenize	#14/#8 冗余 prefill 3.8× 、新鲜追加仅 19% [56] ； 命中率→ 0.99 时 tokenize 占 TTFT 64% [68]	每轮 O(上下文) 的 JSON 全量编码 + 全文 BPE （串行、哈希 / 分支密集， 2–4 MB/s/ 核 [68] ）；每步 ~120K token × 8.8 轮 ≈ 4.2 MB/ 请求（推算） ； 人停顿 >5 min 缓存失效 → NPU 重 prefill 。瓶颈：单核 IPC ； 前端核数	S5 增量 tokenize （ 0.5–1.1 ms [68] ） + 增量序列化； tokenized 缓存	H3 单核； H12 每 NPU 前端核数；（ DPU/ 前端池承接）
工具调用 → MCP （ JSON-RPC ）	#7/#2 Codex runner 残余 18.6% （均值 1.11 s 、 p99 10 s ） [56] ； 每会话数十 MB 序列化 [57]	MCP ： 每条消息一次新的 HTTP POST 、 SSE 返回、二进制 base64 （ +33% ） [74] ； 工具结果（文件、搜索、截图）数十 KB–MB ； 每次都走网关 /JSON/ 内核栈。瓶颈：前端 + 解析 + 内核栈	S3 长连接 / 批量帧； S2 二进制帧、 fp16 embedding ； U10 Agent↔ 工具 / 沙箱链路走 UB 消息语义； U8 UM 自举免 TCP 握手	H6 变换引擎（ JSON ）； H1
等人 → 空闲驻留（ KV / 沙箱 / 会话状态）	#6 会话中位 80% 时间空闲； 5 min TTL 驱逐占 31.5% 费用 [57] ； KV 复用 >80% [67]	KV 占 HBM/DRAM 但不计算；驱逐后要重 prefill （ NPU 税） ； 沙箱内存与文件系统状态要保留以便恢复。瓶颈： DRAM 容量、分层介质、恢复延迟	S7 空闲会话卸载（收益待测；受控实验里卸闲置沙箱 − 4.6× [57] ）、 TTL 延长到覆盖 1–10 min 停顿、 zswap 压空闲状态	H4 大 DRAM / UB 内存池； H7 zswap 式内存压缩（ IAA +62–67% [74] ）； H11
LLM 调用 → 走 ⑥	#10/#1/#9	见 ⑥ 的推导链	S6 S7	H9 H5 H2 H3

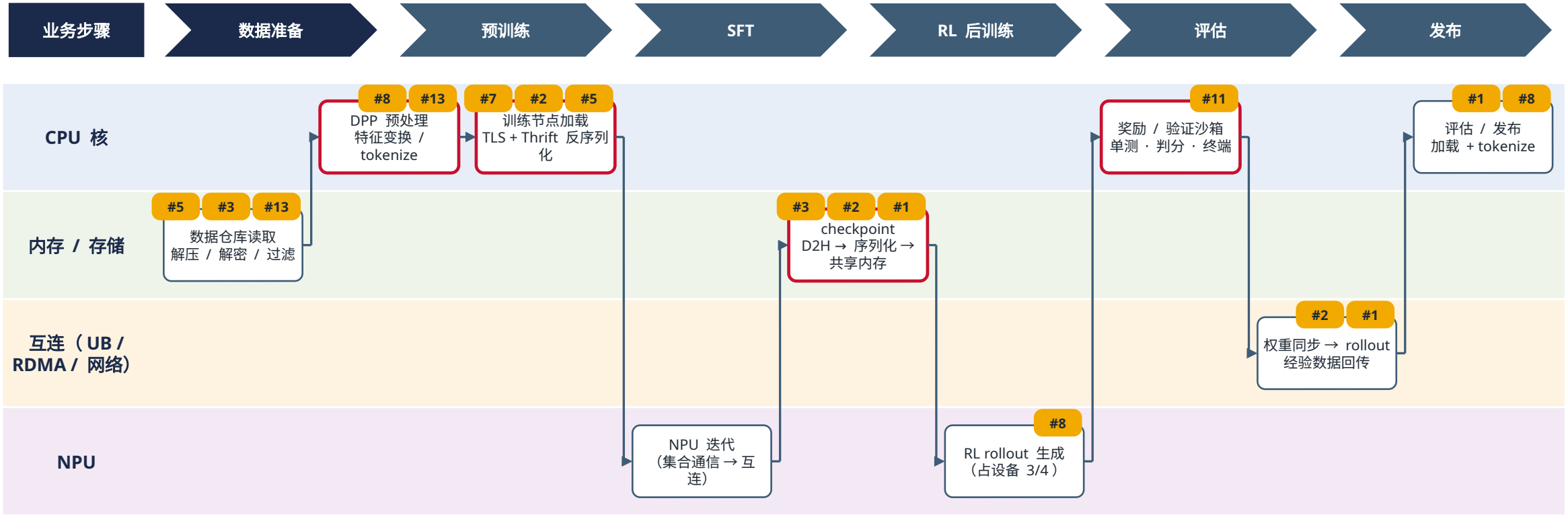
从业务到硅的一条完整推导链

" 跑测试 " 这个业务动作在 Agent 里比 " 生成代码 " 更耗时（ 59.8% vs 41% ）→ 它的成本是进程创建、 syscall 、文件 I/O 被隔离层放大（ gVisor 2.2×–216× 、快照缺页 95% ）加上 28 GB 的峰值内存 → 这些都落在虚拟化路径和内存容量上，不落在任何加速器上，所以编码 Agent 客户要的鲲鹏是：每核能装多少个沙箱、创建 / 恢复多快、机密计算多贵（ H8 、 H2 、 H12 ），软件上先用进程级沙箱 + 快照预取把创建税消掉（ S8 ）→ 第二大税是每轮全量重处理上下文（ 4.2 MB/ 请求）落在单核上（ H3 、 S5 ）→ " 等人 " 的 80% 空闲让状态层需要一个便宜的停靠层（ H4/H7/S7 ）。 RL rollout 是这个循环的批量版，沙箱创建最频繁，见 ⑤。

业务：模型厂商 / 训练平台 · 预训练 + SFT + RL 后训练 · host = 数据 + 状态 + 沙箱

场景 ⑤ 大模型训练与后训练平台：从数据准备到发布的一个作业周期

MFU 55.2% （MegaScale 175B/12K GPU [71]）；预训练作业只占 3.2% 数量却用 94% GPU 时 [62]； RL rollout 占设备 3/4 [76]



业务指标

MFU； checkpoint 停顿（175B/4096 GPU 到 HDFS 200 s/ 次 [63]，同步拖慢最多 43% [62]）； rollout 吞吐（容器环境准备占 19–29% [76]）；每轨迹成本；评估作业 29.5% 时间在加载 + tokenize [62]

关键刻画

两面证据： Meta 训练节点仅 " 加载预处理好的张量 " 就占 40% CPU + 55% 内存带宽，其中是 TLS 解密、 Thrift 反序列化等 datacenter tax， TLS 放大内存带宽 3× [53]；而上海 AI Lab 集群里 CPU:GPU = 16:1 低利用、 CPU 内存 <50% 、 NIC 60% 空闲 [62]—— 数据层紧、训练节点闲。 RL 让 Agent 的沙箱创建最频繁：无预热池、无空闲、万级并发、用完即弃 [76]。

场景 ⑤ 税表： A 对训练进度 / rollout 吞吐的影响 · B CPU 侧资源

税	A： 对进度 / 吞吐的影响	B： CPU 侧资源占比	标注
#3/#1/#2 checkpoint	175B/4,096 GPU 保存到 HDFS 200 s，远超一次迭代 [63]； 同步 checkpoint 拖慢 43% [62]； 异步化后停顿 − 12–161× [63]	D2H → 序列化 → /dev/shm → 上传全在 host； 单文件 2–3 GB/s 上限来自 CPU 序列化而非 NIC [63]； CPU 内存要容纳几个 checkpoint [62]	已发生 · 实测
#13/#5/#2 数据层	data stall 直接停 GPU； 评估作业 29.5% 在加载 + tokenize [62]； 预处理吞吐需求两年 +3.5× [53]	仅加载占训练节点 40% CPU + 55% 内存带宽 [53]； 存储 + 预处理功耗可超训练器 [53]； TLS 放大内存带宽 3× [53]	已发生 · 实测
#10 launch / straggler	42.5% 作业因 straggler 至少慢 10% [71]； CPU 侧操作未被掩盖 → launch 延迟 [71]	CUDA 分配器碎片导致 cudaMalloc/Free 激增 [71]	已发生 · 实测
#11 RL 沙箱创建 / 验证	容器化 SWE 环境准备约 89 s， 64–256 并行时占 rollout 19–29%； 进程级隔离后 23.6 s、占 8–9% [76]； 同步 RL 等最长生成，异步后 2.77× [76]	每任务镜像 1.2–2.5 GB → 进程级存储约 5%； ASandbox 100 ms / 5,000 QPS （runsc/kata） [76]； 奖励验证 = 纯 CPU 可并行 kernel	已发生 · 实测
#1 权重同步 / 经验数据	rollout 等权重的时间可降 47% （Laminar） [76]	trainer → rollout GB–TB/ 步走 NPU 网络， CPU 不在路径；轨迹 /logprob/reward 序列化搬移 [76]	已发生 · 实测
配比	每 rollout NPU 需要的 CPU 核（沙箱 + 前端）〔待内部确认〕	训练节点 CPU 16:1 低利用 vs 数据层紧张 [62][53]	缺

读法

预训练的 host 税表现为 "GPU 停下来等 "（checkpoint、 data stall、 straggler），软件已把大头异步化， B 口径下数据层的 CPU 与内存带宽真实紧张且成分正是老税； RL 后训练让沙箱创建最频繁，沙箱创建延迟与每节点密度直接换算成训练吞吐（环境准备占 rollout 19–29% → 8–9%）。训练节点闲、数据层与沙箱忙—— CloudMatrix 6.1.3 "CPU 物理池化、按需配比 " 的动机 [44]。

场景 ⑤ 推导链：从 " 数据准备 / checkpoint / RL" 到 " 数据层 in-line 、沙箱密度 "

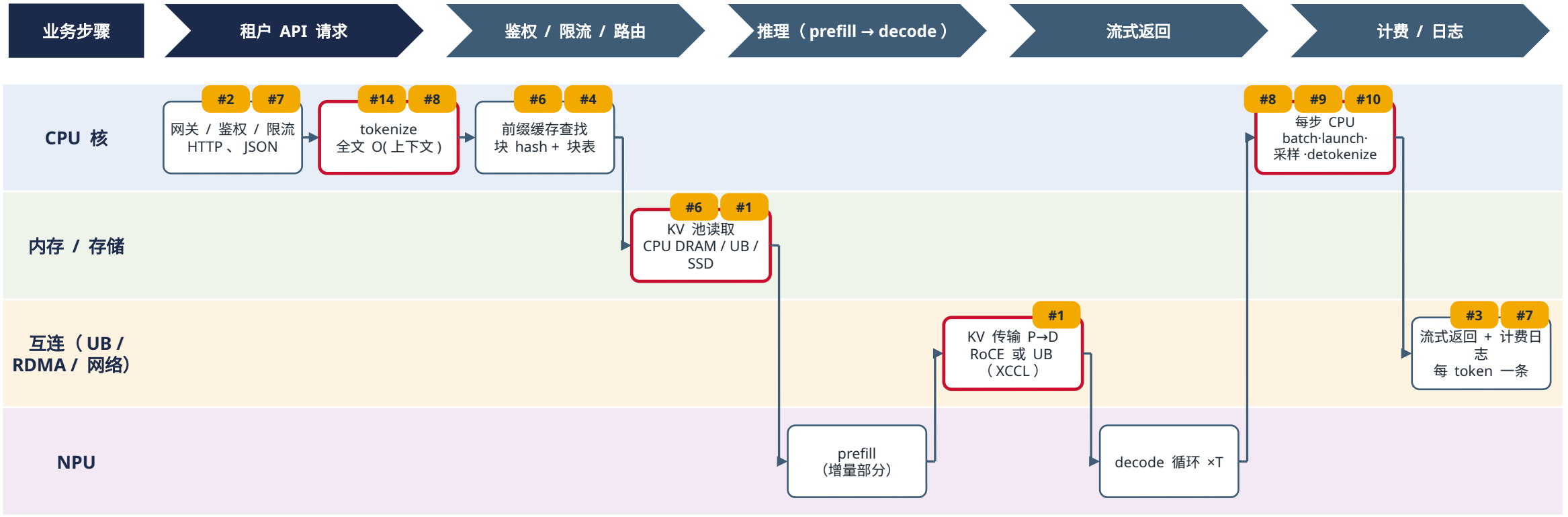
业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950 ）	硬件发力点（→ 960 / SKU ）
数据准备 → 读取 / 解压 / 解密 / 特征变换（ DPP ）	#13/#3/#5 存储 + 预处理功耗可超训练器；预处理需求两年 +3.5× [53]	解压、解密、过滤、变换是大块流式，每一步把数据过一遍内存； TLS 让内存带宽放大 3× [53] 。瓶颈：内存带宽 + 核	S9 KAE in-line 接 DPP/ 加载路径； DPP 与训练节点分离（ Meta 做法）	H7 in-line zip/sec （不额外过内存）； H4 带宽； H12 CPU 密集节点（ CloudMatrix 6.1.3 [44] ）
预训练 → 训练节点加载张量	#5/#2/#7 仅加载占训练节点 40% CPU + 55% 内存带宽 [53]	TLS 解密 + Thrift 反序列化 + 网络栈 + 内存管理——论文原话的 datacenter tax ；内存带宽 ~70% 即饱和 [53] 。瓶颈：带宽 + 前端	S2 零拷贝反序列化； S3 内核旁路 / UB 直达	H7 sec in-line ； H6 变换引擎； H5 搬移引擎
预训练 → 周期 checkpoint	#1/#2/#3 175B/4,096 GPU 到 HDFS 200 s/ 次 [63] ； 同步拖慢 43% [62] ； 异步后停顿 − 12–161× [63]	D2H 拷贝 → pickle/ 序列化 → 落 /dev/shm → 上传；单文件 2–3 GB/s 上限来自 CPU 序列化而非 NIC [63] ； CPU 内存要容纳几个 checkpoint [62] 。瓶颈：拷贝 + 序列化（核） + DRAM 容量	S9 zip 接 checkpoint 流水；异步流水（ ByteCheckpoint 式）	H5 搬移引擎（ D2H/H2D ）； H4 DRAM 容量； H11 checkpoint 落 UB 内存池（ EMS 5 s vs 281 s 同类 [44] ）
RL 后训练 → 奖励 / 验证沙箱	#11 容器环境准备占 rollout 19–29% → 进程级后 8–9% （ 89 s → 24 s ） [76] ； ASandbox 100 ms / 5,000 QPS [76]	create-on-demand / destroy-after-use 、万级并发、无预热无空闲：容器环境准备约 89 s 、每任务镜像 1.2–2.5 GB ；进程级（ namespace + chroot + 预缓存 venv ）后约 24 s 、存储约 5% ，高并行时卡在并发解压 [76] 。瓶颈：虚拟化路径 + 内存 + 密度（与 ④ 同机理、创建更频繁）	S8 进程级沙箱 + 预缓存依赖 + 镜像缓存 /fork	H8 沙箱硬件辅助； H12 高密度款； H7 镜像解压
RL → rollout 生成 → 权重同步 / 经验回传	#1/#2 rollout 占设备 3/4 [76] ； rollout 等权重同步的时间可降 47% [76]	权重 GB–TB/ 步走 NPU 间网络， CPU 不在路径；轨迹 /logprob/reward 在 rollout 与 trainer 间序列化搬移；每条轨迹 tokenize/detokenize 。瓶颈： NPU 互连 + 序列化（核）	S6 rollout 引擎 host 路径； S7 权重 / 经验走 UB 共享内存（ XCCL 路径 [60] ， U9 ）	H11 UB ； H9 提交路径； H6
评估 / 发布 → 模型加载 + 数据 tokenize	#8/#1 评估作业 29.5% 时间在加载 + tokenize [62] ；模型切换 EMS 5 s vs OBS 2,560 s [44]	全量 tokenize 评估集、从对象存储加载权重；训练节点 CPU 16:1 低利用、 DRAM <50% 、 NIC 60% 空闲 [62] 。瓶颈：容量与配比错位	S5 tokenized 缓存； S7 模型缓存（ EMS 类）	H4 / H11 UB 模型缓存； H12 按数据层 / 训练节点 / 沙箱分开配比

从业务到硅的一条完整推导链

" 数据准备 " 和 "checkpoint" 是训练平台里 GPU 停下来等 CPU 的两个业务动作 → 它们的成分恰是 Kanev 的老税（ TLS 、 Thrift 、解压、拷贝、序列化），只是搬到了每 GPU 200 Gbps 的数据流上、每一步都过一遍内存 → 所以数据层的硬约束是内存带宽， KAE 的 zip/sec 只有做成 in-line 、不额外过内存才有收益（ H7 ），搬移引擎和 DRAM 容量对 checkpoint 直接有效（ H5 、 H4 ）→ RL 后训练把 ④ 的沙箱创建推到最频繁（容器环境准备占 rollout 19–29% ），沙箱创建延迟与密度直接换算成训练吞吐（ H8 、 H12 ）→ 而训练节点本身的 CPU 是闲的（ 16:1 、 DRAM <50% ），这就是 "CPU 物理池化、按需配比 " 的依据（ H12 ）。

场景 ⑥ 模型即服务（MaaS API）：一次租户请求的推理路径

生产请求：Azure coding 中位 1,500/13 、 conversation 1,020/129 token [46]； Kimi 均值 7,590/182 [45]； CloudMatrix prefill 6,688 / decode 1,943 tok/s/NPU [44]



业务指标

每 token 成本； TTFT / TPOT SLO ； 每 NPU 并发数； CloudMatrix decode 538 tok/s/NPU@<15 ms TPOT [44]； xDeepServe 峰值 2,400 tok/s/910C @~50 ms [60]； EMS 命中 90% 时 TTFT −59% [44]

关键刻画

每 token KV （bf16）： Llama-3-70B 320 KB 、 Qwen3-235B 188 KB 、 DeepSeek-V3 （MLA） 69 KB （推算）； 8K prompt = 0.54–2.5 GB ； KV 池命中率 30→50% 后饱和、 >50% 块零复用 [45]。 CPU 在每个 decode step 都要回来一次；加速器越快、缓存越好， CPU 前端（tokenize 、调度）在 TTFT 里的占比越高 [68][69][49]。 KV 池是旁路： EMS 让 NPU 直读鲲鹏 DRAM 、不经 CPU 核 [44]。

场景 ⑥ 税表： A 端到端时延（ TTFT / TPOT ） / B CPU 侧资源

税	A： 端到端时延占比	B： CPU 侧资源占比	标注
#8/#14 tokenize	命中率→ 0.99 时 tokenization 占 TTFT 10%→64% [68]； 16 核无状态前端 40 req/s 饱和 [68]； CPU 欠配 TTFT 1.47–7.11× 、引擎超时 [69]	HF ≈2–4 MB/s/ 核（ 1M 字符 ~430 ms ） [68]； 中位每次只追加 1.4K 字符却全量重算 [68]	已发生 · 实测
#10/#9 host 调度	vLLM V0 每步 CPU ~50% （ A100/8B ）、 55% （ 1.3B ）； SGLang ≤18% [49]； graph dispatch 0.6–0.8 ms [44]； 共置干扰 TTFT +139× [58]； GH200 CPU-bound 到 4× 大 batch [59]	构成 = 张量准备 + detokenize + Python 元数据 [49]； 页表遍历占 LLC stall 85% [58]； Grace launch 2.77 µs vs x86 2.26–2.37 [59]； 软件基线 vLLM V1 +1.7× [74]	已发生 · 实测
#1 KV 传输 P→D	逐层流水后 <7% prompt 时间、非重叠 5–8 ms 、 E2E +0.8%； 串行则第二 token +64% [46]； 10 rps@66B 需 90 Gbps [47]	字节不经 CPU 核： GPUDirect RDMA [45] 、 RoCE 平面 [44] 、 UB/XCCL <20 µs@<1 MB [60] 、 GPU 自发起拷贝 [49]	已发生 · 实测
#6/#1 KV 池	EMS over UB vs VPC： prefill +1.52× ， 50%/90% 复用率 TTFT −34%/−59% [44]； 存 KV 比重算 batch +30% 、 GPU −87% （ CXL ROI ） [70]	2 TB DRAM ≈ 3,100 万 token ≈ 310 个 100K 会话（ DS-V3 ， 推算）； 块 hash + 前缀 hash 、多粒度分配、 LRU [44][45]	已发生 · 实测 + 推算
#7 流式 / 计费	每 token 一条小消息（推算，见 ③）	10 万流 ≈ 10–25 核（推算）	推算

读法

A 口径下 MaaS 的税集中在 TTFT 的 CPU 前端（ tokenize 、调度）——加速器越快、缓存越好，占比越高； KV 传输本身可被流水藏住。 B 口径下 KV 池是 "GB" 而非 "cycles" 的事， CPU DRAM 容量直接换算成托管会话数与 NPU 算力节省。③④ 的每一次 LLM 调用都承担这张表。

场景 ⑥ 推导链：从 " 一次 API 请求的 TTFT" 到 " 单核、 TLB 与前端核数 "

业务步骤 → 系统环节	税（数字）	机理 / 瓶颈资源（为什么、卡在哪）	软件发力点（→ 950 ）	硬件发力点（→ 960 / SKU ）
租户请求 → 前端 tokenize	#8/#14 命中率→ 0.99 时 tokenize 占 TTFT 10%→64% [68]； 16 核无状态前端 40 req/s 饱和 [68]； CPU 欠配 TTFT 1.47–7.11× [69]	BPE = 正则预分词 + 哈希查找 + 合并优先队列，逐序列串行、分支 / 哈希密集（ HF 2–4 MB/s/ 核）；前端对全文重算而 NPU 只算增量 → 前缀缓存越好它占比越高。瓶颈：单核 IPC + 前端核数； SIMD 帮助有限	S5 增量 tokenize （ TokTier 0.5–1.1 ms ）、 tokenized 缓存、前端 CPU 池 / DPU 承接	H3 单核 IPC； H12 每 NPU 前端核数（ [69]： 加核是最便宜的缓解）
推理 → 每个 decode step 回到 host （组 batch / launch / 采样 / detokenize ）	#10/#9 vLLM V0 每步 CPU ~50% （ A100/8B ） [49]；昇腾 graph dispatch 0.6–0.8 ms [44]；共置干扰 TTFT +139× [58]	Python 元数据 + 张量准备 + detokenize 是分支 / 指针密集的单线程代码；页表遍历占 LLC stall 85% （碎片化虚拟地址空间），共置让 TLB miss 更贵、 dispatch 间隙 1→40 μs [58]； Arm 核 launch 比 x86 慢 17–22%、 GPU 越快 CPU-bound 区越大 [59]。瓶颈： TLB/LLC/ 单核 IPC + CPU–NPU 链路延迟；对共置极敏感	S6 graph 模式、异步调度（ vLLM V1 +1.7×、 SGLang 零开销）、采样 / 元数据上 NPU [44]、 host 与沙箱分池； S4 大页	H2 更大更快 L2 TLB 、页表遍历缓存； H3 单核； H9 UB 内存语义提交队列 / 低延迟 doorbell
推理 → KV 传输 P→D	#1 逐层流水后 <7% prompt 时间、非重叠 5–8 ms [46]； 10 rps@66B 需 90 Gbps [47]	GB 级 KV 若经 CPU 拷贝就是 memcpy 税 × 三个数量级；生产系统一律绕过 CPU （ GPUDirect [45]、 RoCE 平面 [44]、 UB/XCCL <20 μs@<1 MB [60] ）。瓶颈：互连带宽 / 延迟，不是核	S7 实测 Mooncake kunpeng_transport / ubshmem [75] （ U9 ）	H11 UB CPU 出口带宽（ x32 lane ）与远端读延迟（ U9 ）； H5 搬移引擎
推理 → 前缀查找 + KV 池（ CPU DRAM / UB / SSD ）	#6/#1 EMS over UB vs VPC： prefill +1.52×、 TTFT −34%/−59% [44]；命中率 30→50% 后饱和 [45]	块 hash + 前缀 hash 、多粒度分配、 LRU 、 SSD 分层——通用系统软件； NPU 直读鲲鹏 DRAM 不经 CPU [44]； 2 TB ≈ 310 个 100K 会话（推算）。瓶颈： DRAM 容量 + 互连语义 + 热块复制	S7 EMS 类参考架构、热块复制、 TTL （ U9 ）； S4 块表分配	H4 大 DRAM / 通道； H11 UB 内存语义（ cache-line 按需取 2.1× [70] ）； H7 内存侧位平面变换 + codec （ KV 无损 46.9% [70] ）
流式返回 → 计费 / 日志	#7/#3 每 token 一条小消息（推算）；日志压缩	同 ③ 的流式机理；计费日志是大块压缩	S3 批量帧 / kTLS； S9 zip 接日志	H1 前端； H7

从业务到硅的一条完整推导链

" 一次 API 请求 " 的 TTFT = 前端 tokenize + 前缀查找 + 调度 + prefill → NPU 越快、缓存命中越高，留在 CPU 上的 tokenize 与调度占比越大（ tokenize 可到 64% ， CPU 欠配时 TTFT 高 7× ）→ 这两项都是分支 / 哈希 / 指针密集的单线程代码，卡在单核 IPC 、 TLB/LLC 和 CPU→NPU 的提交延迟上，所以 MaaS 客户要的鲲鹏是：单核（ H3 ）、 TLB （ H2 ）、提交路径（ H9 ）和每 NPU 配多少前端核（ H12 ），软件上先把增量 tokenize 与 graph/ 异步调度做进去（ S5 、 S6 ）→ KV 传输与 KV 池是 "GB 与互连语义 " 的事：字节绕过 CPU 核、 CPU DRAM 作状态层（ H4 、 H11 、 S7 ），压缩只在内存侧加变换后才有效（ H7 ）。业界 "CPU-free" 路线（ Blink ）若成主流，前端与状态层就是鲲鹏在推理集群里剩下的价值。

03

综合：汇总矩阵 → 痛点榜 → 动作清单 → 待确认

六个业务场景叠起来看：哪些税反复出现、量最大、最能被一块硅一次性吃掉

综合 1/8 · A/B 口径混排（各场景税表已分列）； "—" = 该场景不出现或可忽略

完整汇总矩阵：税种 × 业务场景，格内为主要数字与出处

税种	① 电商交易	② 信息流推荐	③ 智能客服 RAG	④ 编码 Agent	⑤ 训练与后训练平台	⑥ MaaS 推理
#1 拷贝 / 搬移	memcpy 4–5% cyc [01]	embedding 表 gather 主导 [72]	embedding 载荷（推算）	数十 MB/ 会话 [57]	加载占 55% 内存带宽 [53]； ckpt D2H； 权重同步 [76]	KV 传输 <7% （流水） [46]
#2 序列化 / 解析	protobuf 3–4% [01]； 轻服务 40–90% [13]	级联特征传递；单阶段后 OPEX 10.6% [77]	引擎外 30–50% [66]	>20 MB embedding/ 请求 [57]	Thrift （含于 40% CPU ） [53]	tensor （小）
#3 压缩	5–6% [01]； RPC 3.1% [02]	日志 / 样本回流	—	镜像解压 +40 ms [51]	ckpt 200 s [63]； 解压 [53]； 每任务镜像 1.2–2.5 GB [76]	KV 直接压无效 [48][70]
#4 哈希	1–2% [01]	—	—	块 hash [44][45]	—	块 hash + 前缀 hash [44][45]
#5 加密	<0.2% [02]	—	流式 record （推算）	（机密计算）	TLS 放大内存带宽 3× [53]	（跨节点 KV ，推算）
#6 分配 / 状态驻留	6–7% [01]	embedding 表 TB 级 [72]	索引节点（推算）	中位 80% 空闲； TTL 驱逐 31.5% 费用 [57]	ckpt 暂存； DRAM <50% 利用 [62]	块表； 2 TB ≈ 310 会话（推算）
#7 网络栈 /RPC	均值 2% 尾 96% [02]； 内核 18–20% [01]	级联多阶段 RPC （推算）	引擎外 30–50% [66]； 流式（推算）	runner 残余 18.6% [56]； MCP 每消息一 POST [74]	NIC 60% 空闲 [62]	流式每 token （推算）
#8 tokenize	—	生成式推荐序列 token （趋势）	（生成走 ⑥）	冗余 prefill 3.8× [56]	评估 29.5% 加载 +tokenize [62]； 每轨迹固定	TTFT 10→64% [68]； 欠配 7.11× [69]
#9 采样 / 约束解码	—	—	—	（同 ⑥）	（ rollout 同 ⑥）	CPU 采样 → NPU [44]； XGrammar [73]
#10 调度 /launch	scheduler 5% [01]	—	—	（同 ⑥）	straggler launch 延迟 [71]	V0 ~50% [49]； 0.6–0.8 ms [44]； 共置 139× [58]
#11 沙箱	—	—	（工具调用）	工具执行 59.8% [56]； 0.8/28 GB [57]	RL 环境准备 19–29%→8–9% [76]	—
#12 向量距离	—	ANN 召回，带宽型 [52][65]	<1%–86% [52]； 带宽型	（检索型 Agent ）	—	—
#13 数据层	—	特征流水线（推算）	—	—	40% CPU + 55% 带宽 [53]； 功耗可超训练 [53]	—
#14 O(上下文) 重处理	—	—	（ prompt 拼装）	4.2 MB/ 请求（推算）	（推算）	全文重 tokenize [68]

实测 = 文献数字；推算 = 本文算式（输入见各场景页与 INSIGHT_AI_WORKLOAD.md 第 7 节）。同一税在不同场景的量级差异本身就是结论：拷贝、序列化、分配这三项老税在六个场景里都出现；② 的 AI 化（生成式推荐）正把推荐的税向 ⑥ 合并。

综合 2/8 · A 口径排序（p4），B 口径排序在右下角；括号内为主要业务场景；"谁能吃掉"给出 S/H 编号

痛点榜展开：现状 · 软件消灭到什么程度 · 剩余落在哪块硅 · 谁能吃掉

#	痛点	已发生 / 趋势	软件正在消灭到什么程度	剩余部分落在哪块硅	谁能吃掉
1	沙箱 / 工具执行（④⑤）	已发生（三份 trace）	本地 Agent 已进程级（bubblewrap/seccomp） [74]； RL 进程级后 <5% [76]； 快照预取 3.7× [64]	CPU 核 + DRAM （峰值 28 GB） + 虚拟化；云端仍是容器 /microVM	S8 → H8 H2 H12
2	tokenize + 重处理（⑥④）	已发生	增量 tokenize + GPU BPE （TokTier 437–491× [68]）； vLLM V1 挪出核心循环 [74]	CPU 单核（BPE 串行）；核数配比	S5 → H3 H12
3	引擎外编排 RPC / 序列化（③④②）	已发生	服务端编排（Parrot 11.7× [66]）；二进制 /fp16 向量格式	CPU 核（JSON 解析、拷贝）	S3 S2 S1 → H1 H6； U1 U2 U6 U8
4	向量检索 / embedding 表（③②）	已发生	SIMD 已达带宽上限 [52]； LVQ 压缩 [73]	内存带宽；溢出后远端内存粒度 [65]	S10 → H4 H11 H10
5	训练数据层 / checkpoint （⑤）	已发生	异步 checkpoint 3.6–58.7× [62]、停顿 − 12–161× [63]	host CPU + 内存带宽（TLS 3×）	S9 → H7 H5 H4 H12
6	host 调度 / launch （⑥）	已发生，软件在消灭	vLLM V1 +1.7×、SGLang ≤5.6% [74][49]；采样上 NPU [44]； CPU-free [58]	TLB/LLC/ 单核 + CPU–NPU 链路；共置敏感	S6 S4 → H2 H3 H9； U4
7	KV 搬移与状态层（⑥④）	已发生（UB 已在生产）	逐层流水 [46]； EMS/XCCL 绕过 CPU [44][60]	内存容量 + 互连；"存着等"	S7 → H4 H11 H7； U9
8	传统 tax 延续（①→③④⑥）	已发生（十年稳定）	零拷贝序列化 [19][26]；内核旁路	CPU 核	S1 S2 S3 S9 → H1 H5 H6 H7； U1–U7

B 口径（CPU 侧资源）排序

1 状态层容量（GB）· 2 训练数据层（40% CPU/55% 带宽）· 3 传统 tax （22–27% cyc）· 4 序列化 /JSON · 5 检索带宽 · 6 沙箱内存 · 7 host 调度 · 8 KV 搬移（绕过 CPU 后趋零）

两种口径的最大分歧是 KV 搬移（A 小、B 曾大、现在被 UB/RDMA 绕过后趋零）与状态层容量（A 不占时延、B 第一）。S/H 编号见 p34–35， U 编号见 p36–38。

软件发力点清单 S1–S11： 现在就能做，先把税测出来、把能软件消掉的消掉

#	发力点	具体内容	怎么做 / 验收 / 产出物	税 · 场景	依据
S1	前端友好的代码布局	PGO/BOLT 、 text 大页、热路径函数重排，针对 RPC 库 / 序列化 / 内核栈 / 网关这些 " 税代码 "	在 920/950 上对 bRPC/gRPC + protobuf + 内核栈跑 PGO/BOLT + text 大页，测 L1-I MPKI 、前端 stall 、 RPC 每核 Mops 的变化；目标：前端 stall 降 20–30%	#7 #2 #3 · ①③④⑥	[01][05][41]
S2	零拷贝与二进制序列化	zBuffer / FlatBuffers / Arrow 直接成为 bRPC-UB 的 SGE； embedding 用 fp16 二进制而非 FP32 JSON； messages 增量序列化	把 zBuffer 式零拷贝接进 bRPC over UB （ protobuf 字段直接成 SGE ）； embedding/ 向量 API 改 fp16 二进制；产出：每 RPC 序列化周期与拷贝字节的前后对比	#2 #1 #14 · ①②③④	[19][26][57]
S3	用户态 / 内核旁路网络 + UB 消息语义	bRPC over UB 的小消息 slab 修复（ U1 ）、 RM 默认化（ U2 ）、 UM 自举建连（ U8 ）、 io_uring 、 busy-poll 、 Homa 式接收端驱动；流式 token 批量帧 + kTLS （ U10 ）	修 ubsocket 8 KB 最小块 / UMQ 4 KB slab 的小消息放大；流式 token 用批量帧 + kTLS ；产出：小消息每核 Mops 、 100 Gbps 所需核数（对标 Snap 7–14 核）	#7 · ①③④⑥	[08][31][33] REPORT 6.3
S4	分配器与大页	mimalloc / TCMalloc 调优（ Arm 上 vLLM 2.3× ）、 THP + text 大页、 KV 块表；缓解 Python 栈碎片化地址空间的页表遍历	vLLM/SGLang/MindIE 前端换 mimalloc 、开 THP + text 大页；测 dTLB miss 、页表遍历占 LLC stall 的比例（对标 [58] 的 85% ）	#6 #10 · ①④⑥	[35][74][58]
S5	增量 tokenize 与前端池	TokTier 式增量修复 + tokenized 缓存；每 NPU 前端核数配比；前端 CPU 池 / DPU 承接 tokenize	把 TokTier 式增量修复接进 MindIE/vLLM 前端； tokenized 缓存；测 TTFT 中 tokenize 占比随命中率的曲线；给出每 NPU 前端核数的配比表	#8 #14 · ⑥④③	[68][69]
S6	推理引擎 host 路径	graph 模式、异步调度、采样 / 元数据上 NPU 、绑核与 host- 沙箱分池	在鲲鹏–昇腾上复现 vLLM V1 / SGLang 的异步调度，采样搬到 NPU ；测每 step CPU 时间拆解与 graph dispatch 延迟（对标 0.6–0.8 ms ）	#10 #9 · ⑥⑤	[44][49][60][74]
S7	状态层软件	实测 Mooncake kunpeng_transport / ubshmem （ U9 ）； EMS 类 KV 池参考架构；空闲会话卸载、 TTL 策略、 zswap	同机跑 Mooncake kunpeng_transport vs rdma_transport： Mops 、 GB/s 、 CPU 占用、远端读延迟； EMS 类 KV 池参考架构 + 每节点托管会话数指标； zswap 压空闲会话的收益	#6 #1 · ⑥④②⑤	[44][45][57][75]
S8	沙箱软件栈	进程级沙箱、快照 + 工作集预取（ REAP ）、预热池、 OverlayFS 、 virtCCA 机密 VM	Firecracker / Kata / gVisor / 进程级沙箱在 950 上的创建 · 快照恢复 · 每核密度 ·SMT 取舍基线； virtCCA 机密 VM 的 I/O 开销（对标 <29.7% ）	#11 · ④⑤	[51][64][61][74][76]
S9	KAE 接入 AI 路径	UADK Python/PyTorch 零拷贝绑定； zip 接 checkpoint / 镜像 / zswap ； sec in-line 接训练数据层	UADK 的 Python/PyTorch 零拷贝绑定； zip 接 ByteCheckpoint 式流水与沙箱镜像、 Linux zswap ； sec in-line 接 DPP/ 加载路径；产出： in-line 约束下的实际收益表	#3 #5 #13 · ⑤④⑥①	[53][63][74]
S10	检索与 embedding 表的 SVE / 带宽调优	faiss / SVS / ScaNN 移植； DLRM gather 与 ANN 的带宽利用率基线	faiss / SVS / ScaNN / DLRM 在 950 上的 SVE 移植与带宽利用率（对标 18 GB/s/ 核、 80% ）；决定是否需要专用引擎（预期：不需要）	#12 #1 · ②③	[52][65][72]
S11	剖析与基准工具链	DCPerf 类基线、每 step CPU 时间拆解、共置干扰复现、鲲鹏–昇腾 launch 延迟——决定 H 清单的优先级	DCPerf 类基线 + 每 step CPU 拆解 + 共置干扰复现 + 鲲鹏–昇腾 launch 延迟 + 沙箱基线，形成 " 鲲鹏 AI 税剖析套件 "；这是 H 清单排优先级的唯一依据	支撑全部	[05][49][58][59]

优先级建议（按痛点榜）： S8 沙箱 > S5 增量 tokenize / 前端池 > S3 网络与 UB 消息语义 > S7 状态层 > S6 host 路径 > S9 KAE 接入 > S1/S2/S4 > S10 ； S11 工具链最先做——它决定 H 清单里哪几条真正值得投。

硬件发力点清单 H1–H12： 软件消不掉的部分落在哪块硅，就在那块硅上投

#	发力点	为什么（机理与证据）	需求数字从哪来 / 验收指标	税 · 场景	依据
H1	前端： L1-I / 取指带宽 / BTB / 指令预取	税代码路径长、二进制数百 MB： 前端受限 15–30% 、 L1-I MPKI 7–56 ； 920 L3 取指带宽弱、 64 项 BTB	需求数字来自 S1/S11： 950 的 L1-I/L2/L3 取指带宽与 BTB 规模（待内部确认）； 验收指标： DCPerf Web/Ranking 类负载的前端 stall 、 L1-I MPKI 、 Perf/W （对标 SKU-B −57% 的反例）	#7 #2 #3 · ①③④⑥	[01][05][41] REPORT 6.1
H2	TLB 与页表	更大更快的 L2 TLB 、页表遍历缓存、大页； Python 栈页表遍历占 LLC stall 85% ； 920 L2 TLB 11 周期	需求数字来自 S4/S6： 页表遍历占 LLC stall 的比例、共置下 TLB miss 代价；验收： vLLM 前端在共置干扰下的 TTFT 退化倍数（对标 139× ）	#10 #6 #11 · ⑥④⑤	[58][35] REPORT 6.1
H3	单核 IPC 与频率（关键路径核）	主频差 9% → 尾延迟差 1.5× ； Arm launch 比 x86 慢 17–22% ； BPE 串行 2–4 MB/s/ 核	需求来自 S5/S6/S11： nullKernel launch 延迟（对标 x86 2.3 µs ）、 BPE 每核吞吐、关键路径服务 P99 对频率的敏感度（对标 9%→1.5× ）	#7 #10 #8 · ①⑥④	[06][07][59][68]
H4	内存系统	通道数 / 带宽（ MRDIMM 类）、容量、 UB 内存池、内存侧位平面变换 + codec	需求来自 S7/S10： DLRM/ANN 的带宽利用率、 KV 池每节点托管会话数； SKU： 通道数、 MRDIMM 、容量； UB 内存池的容量 / 延迟；内存侧位平面变换 + codec 的可行性（ TRACE 256 GB/s ）	#6 #12 #1 #13 · ②③⑥⑤	[52][65][70][72]
H5	搬移引擎	DPA （片上搬移加速器）作 DSA 、与 UB 内存语义合一；小拷贝也要划算（片外阈值 ⩾ 425 B ）	需求来自 S2/S3/S7： 小拷贝（ <512 B ）占比与片上引擎的提交延迟； DPA 的用户态接口（低延迟提交 + SG + 与内存池共段）	#1 · ①⑤⑥④	[20][28][44]
H6	结构化数据变换引擎	protobuf + JSON + tensor ，近核 / 片上，异步或链式接口	需求来自 S2： protobuf/JSON 的 KB 级同步调用粒度 → 接口必须异步或链式（ [20] [41] ）； 放置：近核 / 片上（ ProtoAcc RoCC vs Kanev 近 LLC 的争议）	#2 · ①③④②	[25][34][41][57]
H7	KAE zip / sec 演进	KB 级小 buffer 固定开销、 in-line / DMA 直达不额外过内存、 zswap 路径	需求来自 S9： KAE 小 buffer （ KB 级同步）固定开销 vs CDPU 近核； in-line/DMA 直达的数据层路径； zswap 批量压缩接口	#3 #5 #13 · ⑤①④	[34][53][74]
H8	沙箱硬件辅助	stage-2 页表复制、内存清零卸载、快照恢复 DMA 、低开销机密计算、 SMT 隔离	需求来自 S8： 沙箱创建 / 快照恢复的延迟分解（页表、清零、页拷贝）、每核密度、 CCA 开销；目标对标 ASandbox 100 ms / 5,000 QPS 与进程级隔离的环境准备 23.6 s （容器 88.9 s ）	#11 · ④⑤	[51][64][61][76]
H9	CPU → NPU 提交路径	UB 内存语义队列 / doorbell 、低延迟 launch （对标 x86 2.3 µs 、昇腾 graph dispatch 0.6–0.8 ms ）	需求来自 S6： 鲲鹏→昇腾 graph dispatch 与同步延迟 vs PCIe-x86 vs GH200 NVLink-C2C ； UB 内存语义提交队列（ UMQ ）的实测	#10 · ⑥⑤	[44][59][60]
H10	SME2 级矩阵扩展 + BF16/INT8	rerank / embedding / draft / 生成式推荐前端上 CPU ； 生态（ KleidiAI/oneDNN ）已就绪	需求来自 S10 与客户数据： rerank/embedding/draft 模型有多少落在 CPU ； 生态： KleidiAI/oneDNN 对 SVE/SME 的支持	#12 #9 · ③②⑥	[74][77]
H11	UB CPU 出口带宽与远端读延迟	CPU x32 lane vs DRAM 带宽哪端先饱和； cache-line 粒度按需取（稀疏注意力 2.1× ）	需求来自 S7： CPU x32 lane 的实际出口带宽 vs DRAM 带宽； CPU 侧 URMA 远端读延迟 vs NPU 侧 XCCL （ <20 µs@<1 MB ）	#1 #6 · ⑥②④	[54][60][70]
H12	SKU 配比	高密度款（沙箱 / 前端 / 状态层） vs 高单核款（ host ）； 每 NPU 的 CPU 核数； CPU 池化节点	需求来自 S5/S8/S11： 每 NPU 前端核数、每核沙箱数、数据层 vs 训练节点 vs 沙箱池的 CPU/DRAM 配比； CloudMatrix 6.1.3 的 CPU 池化节点 SKU	全部	[44][62][69]

优先级建议： H1 前端与 H2 TLB 是 fleet 平均的最大杠杆（六个场景都受益，且 920 的公开弱点正落在此）； H8 沙箱与 H12 配比是 Agent/RL 时代增量最大的； H4/H11 决定鲲鹏在推理集群里的 " 状态层 " 定位； H5–H7 引擎吃的是各 3–6% 的具体税，收益确定但需 in-line/ 近核接口； H9/H10 取决于 host 是否留在推理路径上（争议 9 ）。

综合 5/8 · 依据 REPORT 6.3 的需求对照表（有 / 部分 / 待测） + 本地 UB 文档与 brpc_ub 代码

UB 专题 1/3：把一次 RPC 的时延拆成八段，每段文献怎么说、 UB 有什么、差在哪

时延链环节	文献量级与最优解	软件基线（现状）	UB 原语〔本地〕	状态	差距 → 发力点
① 序列化 / 组包	拷贝是主体、元数据其次、编码最少 [19][26]； zBuffer 25 ns 、比 Cornflakes 7× [19]	bRPC： protobuf → IOBuf 拷贝式序列化，再挂接 UMQ block 链表	UMQ 预注册 qbuf 池、 SGE 、共享 JFR	有	U6： protobuf 字段直接成 UMQ 的 SGE （软件最便宜的一步）
② 提交（ syscall / doorbell ）	PCIe doorbell 是小消息固定开销； Dagger 用内存互连提交 +14% [12]； eRPC 用户态 10 Mrps/ 核 [31]	用户态 URMA post ，无 syscall	DWQE： st64b 64 B 原子 MMIO 直写整条 SQE ，硬件免 DMA 取描述符； SoC 内无 PCIe 往返	有	U1： 小消息 inline 与 DWQE 配合；每核 Mops 待测（对标 eRPC 10 M 、 Falcon 单 QP 20 M ）
③ 传输（ fabric ）	eRPC 2.3 µs RTT [31]； Falcon 1 B–2 MB 5.8–8.7 µs 、松弛排序 + 多路径比 RoCE 好 8× [30]； CXL 域内 1.5 vs RDMA 17 µs [29]	RoCE： 单路径严格有序、 PFC 依赖	RM 无连接可靠、 ROI/ROT/ROL/UNO 四种排序、 TPG ≤32 通道逐包 spraying + OOR 重排、 RTP 端到端重传 / CTP 机架内无损两档； UB 2.0 超节点 2.1 µs 〔公开〕	有	U2： 默认改 RM + share_tp ； U7： 同域走 OBMM 共享内存 RPC ； RTT 与每 Gbps 核数待测
④ 接收通知	Google 生产中断唤醒 >50 µs 伤尾延迟 [02]； CXL 只能轮询（ MSI 未通） [14]	轮询（浪费核）或中断（慢）	JFCE 完成事件通道、消息中断（中断地址表 / 向量表）、远端事件通知	有 · 量级待测	U5： 通知→唤醒延迟实测；混合轮询策略
⑤ 核间分发	RSS 静态哈希浪费 30–76% [11]； nanoPU JBSQ 有界队列 [10]； Cerebros 按方法亲和 [13]	ubsocket 软件多平面 LB （ affinity_priority / rr ）	Jetty Group： 目标侧多 RQ ，硬件按 Hint 哈希 / RR / RQ 深度选 RQ ，免 CPU 、 NUMA 亲和	有（ RQ 深度）	U4： 有界队列 + 超时回收 / 抢占、按 RPC 方法亲和—— SoC 内 ns 级反馈的结构性机会
⑥ 排队 / 过载 / QoS	Homa 接收端 grant [08]； Breakwater 信用 <20 ms 收敛 [16]； Aequitas 按 RPC 标 QoS fleet −10% [17]； Falcon 四时间戳 CC + 接收侧资源回传 [30]	TCP： 多 RPC 复用连接无法按消息标 QoS ； ubsocket 每链路一个 link_priority	UMQ 应用层信用（授信压进 64 位立即数、独立 FC Jetty 、共享信用池）；事务层 RNR ； SL 0–15 / VL ；传输层 CC	部分	U3： 每消息 SL 映射、 CQE 硬件时间戳、可编程 CC 、接收 NIC 缓冲占用回传（ ncwnd ）—— UB 消息边界 = WR 边界，比 TCP 容易
⑦ 反序列化 / 拷贝	网络栈拷贝 >50–60% CPU [33]； 多数拷贝 <512 B [20]； DDIO/LLC 占用	ubsocket 最小块 8 KB 、 UMQ qbuf 4 KB slab + 512 B headroom 对中位 <1.5 KB 的 RPC 是内存放大〔本地〕	零拷贝 block 链表挂接（非 POSIX SG ）； UB Controller 是否直写 L3 待测	有 · 小消息放大	U1： <256 B inline 、小消息专用 slab ； U6： non-pin SVA 免 pinned 簿记
⑧ 连接 / 权限 / 建连	RC QP 状态溢出 NIC cache 吞吐骤降 [31][38]； MR 注册走内核 [26]； TLS 建连 1.6 ms [21]	brpc_ub 默认 RC_TP 一对一独占； TCP 握手交换 EID/Jetty 再切 UMQ ，失败逃生回 TCP	RM： TP 按节点对共享（ O(节点^2) ）、 MUE 托管上下文； MAPT 用户态权限队列（ PERMQ ）、 Token 可转授、 non-pin 段； UM 带内自举	有 · 默认未开	U2 默认 RM ； U8 UM 自举替代 TCP 握手；连接数 vs 延迟曲线待测（对标 Falcon >100K 连接无悬崖）

读法：①②⑦ 是 CPU 侧的税（前端 + 拷贝）， UB 的贡献是把它们的软件路径缩短（ DWQE 、预注册、零拷贝接缝）；③–⑥ 是互连 /NIC 侧的税， UB 在传输层已有文献要求的机制，缺的是 "RPC 感知 " （按消息 QoS 、时间戳、有界分发）；⑧ 是默认配置把卖点关掉了。 Falcon 的硬件预算可作参照：接收侧 O(BDP) 包缓冲 1–2 mm^2 ，传输 IP 面积 5–6% / 功耗 3–4% [30] 。

UB 专题 2/3：发力点 U1–U10——RPC 时延链 + AI 数据路径

#	发力点	为什么（机理与证据）	税 · 场景	依据	标签
U1	小消息路径：inline 发送 + 小 slab + DWQE 直写	中位 RPC <1.5 KB [02]，而 ubsocket 最小块 8 KB、UMQ qbuf 最小 4 KB slab + 512 B headroom〔本地〕：内存放大与 LLC 占用；eRPC/Falcon 对 <256 B 走 inline；DWQE 用 st64b 把整条 SQE 直写设备、免 doorbell 往返（Dagger：内存互连提交 +14% [12]）	#7 #1 · ①③④⑥	[02][12][30][31] REPORT 6.3	→950 软件 + 待测
U2	无连接 RM 模式默认化 + TP 通道共享	bRPC-UB 默认 RC_TP 一对一独占 TP（"同 IB RC 的扩展性问题"〔本地〕）；RDMA RC 的 QP 状态随连接数溢出 NIC cache 吞吐骤降 [31][38]；ServiceRouter 型客户端连数万后端 [21]。RM 模式每条 WR 自带 EID/Jetty、TP 按节点对共享（O(节点²)）——这是 UB 相对 RoCE 的核心卖点，不该在默认配置里放弃	#7 · ①③④	[21][31][38] REPORT 6.3	→950 软件
U3	RPC 感知的传输：每消息 SL/VL、CQE 时间戳、可编程 CC、接收侧资源回传	Aequitas 按 RPC 标 QoS fleet −10%，多 RPC 复用连接是障碍 [17]——UB 消息边界 = WR 边界，天然可做；Falcon 的四时间戳延迟 CC + 接收 NIC Rx 缓冲占用率回传（ncwnd）处理 PCIe/IOMMU/ 内存瓶颈 [30]；ubsocket 当前每链路一个 link_priority〔本地〕	#7 尾延迟 · ①③⑥	[17][30][02] REPORT 6.3	→960 UB Controller
U4	NI 侧硬件分发：Jetty Group → 有界队列 + 按方法亲和	RSS 静态哈希浪费 30–76% 吞吐 [11]；Jetty Group 已按 RQ 深度选队列（RPCValet 式 NI 分发）〔本地〕；差距是 nanoPU JBSQ 的有界队列 + 超时回收 / 抢占 [10] 与 Cerebros 的按 RPC 方法亲和 [13]；SoC 内 UB Controller 有 ns 级反馈的结构性机会	#7 #10 · ①③⑥	[10][11][13] REPORT 6.3	→960 UB Controller
U5	通知与唤醒：JFCE / 消息中断替代轮询	Google 生产 >50 µs 的唤醒延迟伤尾延迟 [02]，轮询又浪费核；CXL 只能轮询（MSI 未通）[14]；UB 有 JFCE 完成事件通道、消息中断、远端事件通知〔本地〕——量级待测	#7 · 全部	[02][14] REPORT 6.3	待测
U6	零拷贝接缝：zBuffer 反射 → protobuf 字段直接成 UMQ SGE；MAPT/Token 用户态权限 + non-pin SVA	bRPC 仍是 protobuf → IOBuf 拷贝式序列化；zBuffer 的 scatter-gather 反射 25 ns、比 Cornflakes 7×、阿里生产部署 [19] 可直接叠加到 UB；RDMA 的 MR 注册走内核、NIC 只能 DMA pinned 页是 Cornflakes 复杂度的根源 [26]，UB 的 PERMQ/Token/non-pin 段消除它	#2 #1 · ①③④	[19][26] REPORT 6.3	→950 软件（最便宜的一步）
U7	超节点内共享内存 RPC：OBMM 共享 + 软件 Ownership	CXL 域内共享内存 RPC 1.5 µs vs RDMA 17 µs [29]，大消息 96–247× [14]；UB 的域比 CXL 大（超节点 16 节点 /200 m）且多了通知机制；HydraRPC 主动绕开硬件一致性，与 UB "共享 = 软件一致性" 一致 [14]；需要调度器把高频通信服务对放同域，缺 POE/MPK 类沙箱	#7 #1 · ①②③	[14][29] REPORT 6.3	→950 复现 + →960 沙箱
U8	建连与逃生：UM 带内自举替代 TCP 握手 + UB 协商	brpc_ub 建连仍用 TCP 握手交换 EID/Jetty 再切 UMQ，失败逃生回 TCP〔本地〕；对照 ServiceRouter TLS 建连 1.6 ms [21]；短连接 / serverless 形态下每连接固定开销被放大	#7 #5 · ③④	[21] REPORT 6.3	→950 软件
U9	AI 数据路径：KV / 状态层全部走 UB 内存语义	EMS：NPU SDMA 直读鲲鹏 DRAM、prefill +1.52×、TTFT −59% [44]；XCCL：KV 直传 <20 µs@<1 MB、当前非零拷贝 [60]；Mooncake kunpeng_transport 已在主线、无公开性能数据 [75]；稀疏注意力下 cache-line 按需取比 RDMA 整段搬 2.1× [70]；CPU x32 lane 出口带宽 vs DRAM 带宽 [54]	#1 #6 · ⑥④⑤②	[44][60][70][75][54]	→950 实测 + →960 零拷贝 /lane
U10	流式 token 与 Agent 工具调用走 UB 消息语义	每 token 一条 HTTP/2 + TLS record + syscall，有效载荷 2–5%、10 万流 ≈ 10–25 核（推算）；MCP 每消息一次 HTTP POST [74]；超节点内的网关↔推理、Agent↔工具 / 沙箱链路可换成 UMQ 立即数携带的小消息 + 批量帧，把内核栈税从每 token 变成每批	#7 #2 · ③④⑥	[33][74] 推算	趋势 · →950 原型

优先级建议：U9（AI 数据路径实测，三条生产路径一个 CPU 侧数字都没有）> U1/U2/U8（软件就能改的默认配置）> U6（零拷贝接缝，收益已在阿里生产验证）> U3/U4（UB Controller 的 RPC 感知，→ 960）> U7（同域共享内存 RPC，需调度器配合）> U5 > U10（趋势）。

UB 专题 3/3：验证清单——用文献同口径的数字给 UB 立论

测什么	对标口径（文献）	在哪测 / 载体	回答哪个发力点
小消息 RPC：RTT、每核 Mops、每 Gbps 核数	eRPC 2.3 µs RTT、10 Mrps/ 核 [31]；Falcon 单 QP 20 Mops、12 QP 120 Mops、8 B–2 MB 5.8–8.7 µs [30]；Homa/Linux 18 核 @100 Gbps [32]；Snap 7–14 核 [38]	bRPC over UB（brpc_ub）+ HCOM 微基准；64 B / 512 B / 1.5 KB / 16 KB 四档	U1 U6；判断 RPC 场景 UB 相对 RoCE 的实际差距
连接数 vs 延迟：RC_TP vs RM_TP（+share_tp）	Falcon >100K 连接无悬崖；CX-7 >100K 连接 RTT 3× [30]	ServiceRouter 式客户端连 1K / 10K / 100K 对端	U2；决定 bRPC-UB 默认配置
通知→唤醒延迟：JFCE / 消息中断 vs 轮询	Google >50 µs 中断唤醒 [02]；CXL 轮询 [14]	HCOM 收端三种模式对比；P99 与核占用	U5
Load/Store 远端访存延迟（TP Bypass）与 OBMM 共享内存 RPC	CXL 远端 ~200–400 ns；HydraRPC 1.47 µs 真硬件 [14]；RPCool 1.5 µs 模拟 [29]	超节点内两节点 OBMM 共享段；复现 HydraRPC 协议	U7；也回答 KV 池的远端读（U9）
Jetty Group 分发：RQ 深度 vs RSS 哈希 vs 软件 LB	RPCValet 比 RSS +29–76% [11]；nanoPU 有界尾延迟 [10]	多核收端、请求服务时间双峰分布	U4
CQE 时间戳 / CC 可编程性 / 接收侧资源回传是否存在	Falcon：CC 算法可演化是必需，NIC 核每 RTT 算一次（<32 µs）足够 [30]	UB Controller 规格与驱动接口审查	U3
AI 路径：EMS / XCCL / Mooncake kunpeng_transport 的 CPU 侧数字	XCCL <20 µs@<1 MB（NPU 侧）[60]；EMS +1.52× / TTFT −59% [44]；Splitwise 逐层流水 5–8 ms [46]	同机 kunpeng_transport vs rdma_transport（Mooncake bench）；CPU 侧 URMA 读远端 DRAM；x32 lane 出口带宽 vs DRAM 带宽	U9 H11；决定 "CPU DRAM 作状态层 " 的 SKU 参数
流式 token / MCP 走 UB 的原型	每 token 小消息有效载荷 2–5%、10 万流 10–25 核（推算）	网关↔推理实例走 UMQ 立即数 + 批量帧；对比 HTTP/2 + TLS	U10（趋势，先做原型再定）
UB Controller 是否直写 L3 及容量控制；UMMU TLB/PLB miss 代价	DDIO 的 LLC 争用（REPORT A5）；Cornflakes 注册簿记 [26]	微基准 + perf 计数器	U1 U6

这张表与 REPORT 6.4 的 " 待测清单 " 一致；前三项是 UB 在 RPC 场景立论的第一步，第七项是 UB 在 AI 场景立论的第一步。

争议与待内部确认项

KV cache 会被算法压没吗?

MLA 已把每 token KV 压到 Llama 架构的 1/4–1/5，线性 / 稀疏注意力在继续；但 Agent 上下文 20K→200K→1M 反向增长。判断：总量增、每 token 降，"CPU DRAM 作 KV 池 " 稳。

沙箱会更轻吗?

本地已进程级（bubblewrap/seccomp/Landlock） [74]；云端仍容器 /microVM； RL 进程级后 <5% [76]。轻隔离解决创建税，不解决 28 GB 工作集与 I/O 税 [57]。

host 税会被 " 绕过 CPU" 消灭吗?

NPU 侧采样 [44]、graph 模式 [60]、DPU+GPU 持久 kernel [58]—— 会缩小但不消失（连续批处理决策仍在 CPU）； GH200 显示只要 host 在路上， Arm launch 劣势就进延迟 [59]。

压缩在 AI 里还值得投吗?

直接压 KV 无效 [48]；位平面变换 + codec 无损 1.9× [70]； zswap 压空闲状态 +62–67% [74]。结论：值得，但部件在内存侧、前端要有张量变换。

CPU 推理成气候吗?

Neoverse V2 上 BF16 2.7×、INT8 4.8–6.2× [74]； Arm AGI CPU 136×V3。围绕大模型的小模型（rerank/embedding/draft）在 CPU 上可行，取决于客户负载。

待内部确认（本文不替代内部数据）

- 鲲鹏 950： SVE 版本、内存通道数、L3（仅二手来源： 12 ch、 SVE；与 48 TB/32 颗 = 1.5 TB/ 颗自洽）
- UB 各路径延迟与 Mops： EMS、XCCL（<20 μs@<1 MB 为 NPU 侧）、Mooncake kunpeng_transport——CPU 侧 URMA 一个数都没有
- CPU x32 UB lane 的实际出口带宽 vs DRAM 带宽（哪端先饱和）
- 鲲鹏–昇腾 graph dispatch 延迟（对标 0.6–0.8 ms [44]、x86 2.3 μs/launch [59]）
- Agent 流量的 fleet 级占比（三份 trace 都是单位量，不是占比）
- RL 集群每 rollout NPU 需要的 CPU 核（沙箱 + 前端）
- Firecracker/gVisor/ 进程级沙箱在 950 上的创建 · 恢复 · 密度；机密计算开销（对标 virtCCA <29.7%）
- KAE zip 对 bf16 张量的实测压缩率（推算 1.1–1.5×）

没有找到 fleet 级 "AI 时代 datacenter tax 再刻画 " 论文—— [69] 给了原因： AI 时代 CPU 侧的税以加速器空转而非 CPU 周期的形式出现。最接近的是 AgentSysBench 的 " 控制面税 " [57] 与 Blink 的 "serving-stack tax" [58]。

附录

资料索引 · 关键数字速查 · 方法局限

全部笔记见 RPC_PAPERS/notes/ （01–77），洞察正文见 INSIGHT_AI_WORKLOAD.md， RPC/ 微服务综述见 REPORT.md

资料索引：本 PPT 引用的笔记编号

#	论文 / 资料	出处
01	Profiling a WSC	ISCA 2015 Google
02	Cloud-Scale Characterization of RPCs	SOSP 2023 Google
03	Lifting the Veil (Meta microservices)	ATC 2023
04	Alibaba Trace Analysis	SoCC 2021
07	CRISP	ATC 2022 Uber
13	Cerebros	MICRO 2021
18	mRPC	NSDI 2023
19	zBuffer	PPoPP 2026 阿里
20	Accelerometer	ASPLOS 2020 Meta
21	ServiceRouter	OSDI 2023 Meta
25	ProtoAcc	MICRO 2021
26	Cornflakes	SOSP 2023
29	RPCool / Telepathic CXL	arXiv 2024
32/33	Homa Linux / Host network path	ATC 2021 / 2021–25
34	CDPU	ISCA 2023
35	Memory Allocator at WSC	ASPLOS 2024
39	XLB / Alibaba 2025	arXiv 2025–26
41	Profiling Hyperscale Big Data	ISCA 2023 Google
44	CloudMatrix384 Serving	arXiv 2025 华为
45	Mooncake	FAST 2025
46	Splitwise	ISCA 2024
47	DistServe	OSDI 2024
48	CacheGen	SIGCOMM 2024
49	NanoFlow + WukLab	OSDI 2025 / 博客
50	Autellix	arXiv 2025
51	Firecracker	NSDI 2020

#	论文 / 资料	出处
52	RAGO	ISCA 2025
53	Meta DSI	ISCA 2022
54	UB-Mesh	ISCA 2025 华为
56	TraceLab	arXiv 2026-06
57	AgentSysBench	arXiv 2026-08
58	Blink	arXiv 2026-04
59	GH200 刻画	arXiv 2025
60	xDeepServe	arXiv 2025 华为云
61	virtCCA	arXiv 2023 华为
62	Acme	NSDI 2024
63	ByteCheckpoint	NSDI 2025
64	gVisor + REAP	HotCloud 2019 / ASPLOS 2021
65	向量检索第二层内存	arXiv 2024 SJTU
66	Parrot	OSDI 2024
67	SMetric	arXiv 2026-07
68	TokTier	arXiv 2026-07
69	CPU 欠配刻画	arXiv 2026-03
70	CXL 作 KV 层（4 篇）	2024–26
71	训练 host 合集（4 篇）	2024–25
72	推荐推理合集（3 篇）	HPCA/ISCA 2020
73	P2 合集（10 篇）	2023–25
74	网络资料（6 份）	2024–26
75	Mooncake UB 后端	GitHub 2026-09
76	RL 后训练 host 侧	arXiv 2025–26
77	生成式推荐 HSTU / OneRec	ICML 2024 / arXiv 2025

关键数字速查

指标	数值	来源
传统 fleet 税	alloc 6–7 / compress 5–6 / memcpy 4–5 / protobuf 3–4 / rpc 3 / hash 1–2 →22–27% cycles ； 内核 18–20%	[01]
RPC 全路径	7.1% cycles ； 延迟税均值 2% 、尾 96% ； P90 请求 11.8 KB 、 P99 196 KB	[02]
每 token KV （ bf16 ）	Llama-3-70B 320 KB ； Qwen3-235B 188 KB ； DeepSeek-V3 69 KB ； 2 TB ≈ 310 个 100K 会话（ DS-V3 ）	推算
KV 传输	1.13 GB/512 tok@66B ， 10 rps → 90 Gbps ； 逐层流水后 <7% 、 5–8 ms 、 E2E +0.8%	[47][46]
EMS over UB	prefill +1.52× ； TTFT −34% （ 50% 复用） /−59% （ 90% ）； 模型切换 5 s vs 281 s	[44]
host 调度	vLLM V0 ~50% （ A100/8B ）； SGLang ≤18% ； graph dispatch 0.6–0.8 ms ； 共置 TTFT 139× ； Grace launch 2.77 µs vs x86 2.26–2.37	[49][44][58][59]
tokenize	HF 2–4 MB/s/ 核；命中率→ 0.99 时占 TTFT 64% ； 16 核前端 40 req/s 饱和； CPU 欠配 TTFT 1.47–7.11×	[68][69]
Agent 真实 trace	8.8 调用 + 10.8 工具 / 请求、 4.3 min ； 119K/875/214 token/ 步；命中 95.7% 、冗余 prefill 3.8× ； 工具占响应 59.8%	[56]
Agent 生产会话	中位 20% 时间执行； TTL 驱逐 59.4% 会话、占 31.5% 费用；沙箱 0.8/28 GB ；每会话数十 MB 序列化；复用 >80%	[57][67]
沙箱	Firecracker <125 ms 、 <5 MB 、 150 VM/s ； gVisor syscall 2.2× 、 open 216× ；快照 +95% → 预取 3.7× ； virtCCA <29.7%	[51][64][61]
RL rollout	容器环境准备占 19–29% → 进程级 8–9% （ 89 s → 24 s ）； ASandbox 100 ms / 5,000 QPS ； rollout 占设备 3/4	[76]
训练数据层 / checkpoint	加载占 40% CPU + 55% 带宽、 TLS 3× ； checkpoint 200 s@175B/4096 GPU 、同步 − 43% ； CPU 16:1 闲置、 DRAM <50%	[53][63][62]
向量检索	<1%–86% 端到端； PQ 18 GB/s/ 核、 80% 带宽； SSD 放大 5.8–7.7×	[52][65]
推荐（ DLRM → 生成式）	DLRM 占 AI 推理周期 79% 、内存主导、近内存 9.8× ； HSTU 1.5T 、 A/B +12.4% ； OneRec 25% QPS 、 OPEX 10.6% 、 MFU 23.7/28.8%	[72][77]
压缩	CacheGen 比 8-bit 再压 3.5–4.3× （有损）； TRACE 变换后 KV 46.9% （无损）； IAA-zswap +62–67%	[48][70][74]
UB 事实	CPU x32 / NPU x72 lane ； UB 2.0 2.1 µs 、 TB/s ； XCCL <20 µs@<1 MB ； TaiShan 950 SuperPoD 16 节点 /32 颗 /48 TB	[54][60] HC2025

方法局限与标注说明

占比不可跨流程直接相加

每条流程的 A 口径是 " 一次请求 / 一条轨迹 " 内的份额，分母不同； p3 热力图的等级只表示量级，不表示 fleet 加权。 fleet 加权需要各业务的流量占比—— Agent 的 fleet 占比目前没有公开数据。

实测数字有各自的硬件与年代

vLLM V0 的 50% 是 2024 年 A100 数据（V1 已 +1.7×）； Firecracker 是 2019 年 x86；Kanev 是 2015 年 Ivy Bridge。用作量级与方向，不用作绝对值；鲲鹏上的数字全部待测。

推算值给出输入

KV 字节数按公开模型配置；托管会话数按 2 TB / 每 token KV / 100K；流式消息按 [33] 的每条 2–5 µs；重处理量按 TraceLab 中位数 × 步数。输入变则结论变。

三份 Agent trace 的代表性

TraceLab 是 43 名开发者的编码 Agent； AgentSysBench 的生产 trace 是三个应用 24 h；SMetric 是阿里百炼。都不是 fleet 级，但三者独立同向。

华为公开资料的使用

CloudMatrix384、 xDeepServe、 UB-Mesh、 virtCCA 论文与 HC2025 发布按〔公开〕引用；内部有更准数据时以内部为准；本 PPT 不猜测内部规格（标〔待内部确认〕）。

没有做的

没有跑任何实测；没有读 Llama 3 基础设施章节、 MTIA、 Catalyzer/VEE 等付费或被覆盖的 6 篇；没有 fleet 级 AI 税再刻画（不存在）；生成式推荐（HSTU/OneRec）的 host 侧无公开剖析，②的 AI 化部分以趋势标注。